

I-007: Introduction to MCUboot Lab (FSP v4.3.0)

Description

The objective of this lab is to build a fully working solution of Renesas RA bootloader using FSP v4.3.0 on EK-RA6M4 kit. This lab will showcase how to upgrade the user application using swap mode and how to work the image verification of MCUboot. This lab will also show the example of firmware image download that using the XMODEM transfer protocol and USB interface.

Lab Objectives • Gain hands-on experience with Renesas FSP • Learn how to create the bootloader project using MCUboot module, and how to sign the user application project image. • Gain hands-on experience to secure upgrade the application image using various image verification.	Lab Materials • Please verify you have the following materials at your lab station. • Hardware ➢ 1 Renesas EK-RA6M4 kit ➢ 1 micro-B USB cable • Software ➢ Renesas FSP v4.3.0 platform installation which includes: • e² studio 2023-04 • FSP v4.3.0 • GCC ARM Embedded Toolchain Version: 10.3.1.20210824 ➢ Python v3.10.x with MCUboot required components ➢ Terminal Software “Tera Term” • Documentation ➢ Application Note R11AN0497
Skill Level • C language and embedded programming • Usage of the FSP, Smart Configurator and e2studio IDE concepts	Time • 70 minutes

Lab Sections

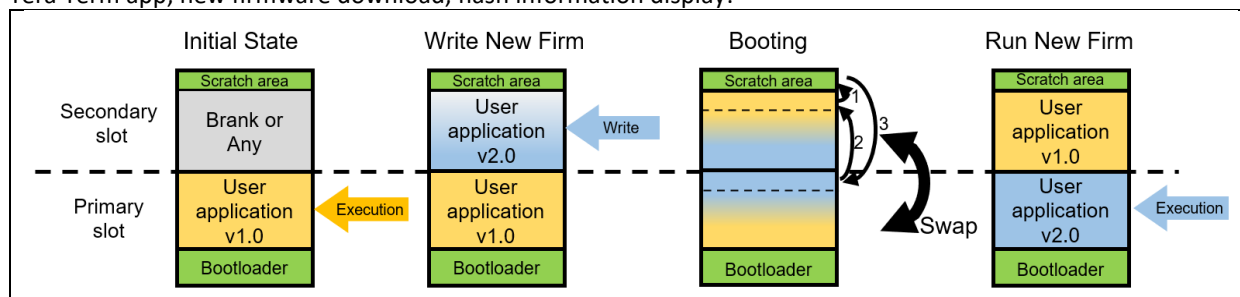
0	Introduction of Lab project	2
1	Creating a Bootloader Project	4
2	Creating an Initial User Application Project	14
3	Hardware Setup.....	25
4	Download Bootloader Application and Initial User Application project	26
5	Upgrading the Application Project	33
6	Testing the revert mechanism of MCUboot Swap test mode	39

0 Introduction of Lab project

This section introduces the overview of the project that will be created in this lab.

In this lab, we will create the bootloader and user application project to demonstrate the MCUboot boot functionality. The following image shows the boot operation on Swap upgrade mode.

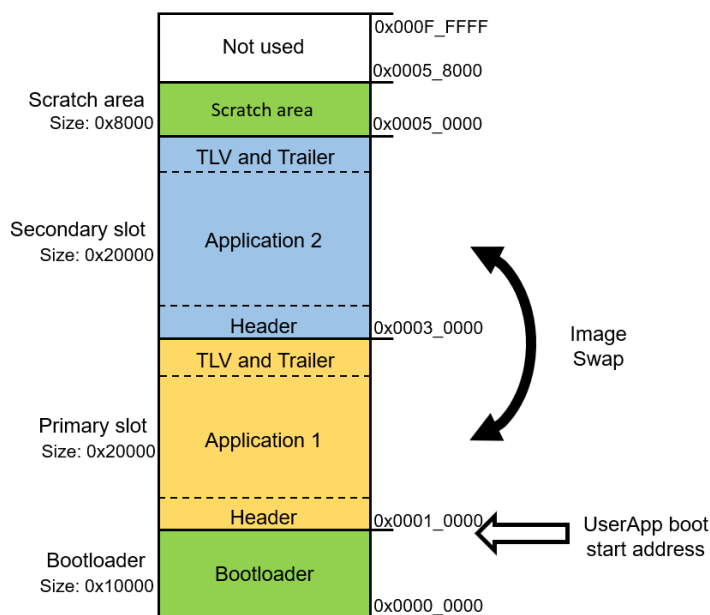
At initial state, we have two images (bootloader and initial user application) in MCU flash, that has been downloaded via debug interface. After reset, bootloader wakes up and boots the initial use application from primary slot, and the communication port of image downloader will open. Now we can download the new firmware from USB interface with XMODEM transfer protocols. After that, we can control the application from Tera Term app, new firmware download, flash information display.



Bootloader Project Overview

Normally, at first, you should determine the bootloader specification (the bootloader upgrade mode, the signature verification used and that options) depends on user's system requirements. Then, determine the flash size for the bootloader area and the primary/secondary image slot area. In this lab, we will use the following bootloader options/configs.

- Update mode: Swap mode
- Image signature verification: Used
 - Signature type: ECDSA P-256
 - Crypt middleware: MbedTLS (Crypto Only)
 - Keys: FSP built-in MCUboot example keys
- Validate Primary Image: Enabled
- Memory map:



User Application Project Overview

This user application project consists of two thread, blinky thread and downloader thread. And this application will run on real-time-operating-system "FreeRTOS".

- **Blinky Thread**
Blinks LED1(Blue) with user-defined blinking frequency (1s period by default).
The source codes will create using FSP's blinky template.
- **Downloader Thread**
Opens the communication port and receive the control commands from Tera Term. If a start command of the image downloading comes, MCU starts an image downloading using XMODEM protocol.
The following LEDs will be used for indication of application status.
 - LED2 (Blue): Turn on during Flash Write/Erase
 - LED3 (Red): Turn on when errors occur.

Project Lists

- MCU4_RA_Bootloader: Bootloader project
- MCU4_RA_UserApp_wDownloader: User Application project
- MCU4_RA_UserApp_wDownloader_new: User Application project (Updated version)

1 Creating a Bootloader Project

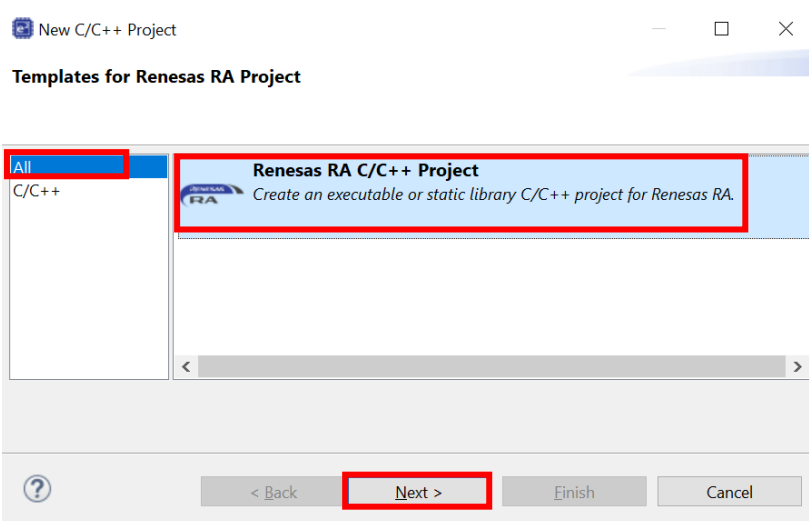
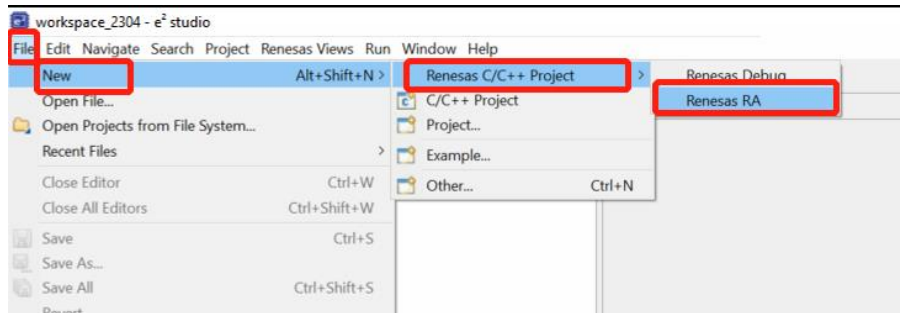
Overview

In this section you will create the bootloader project that will upgrade and boot the user application with the Swap upgrade mode and with the ECC image signature.

Procedural Steps

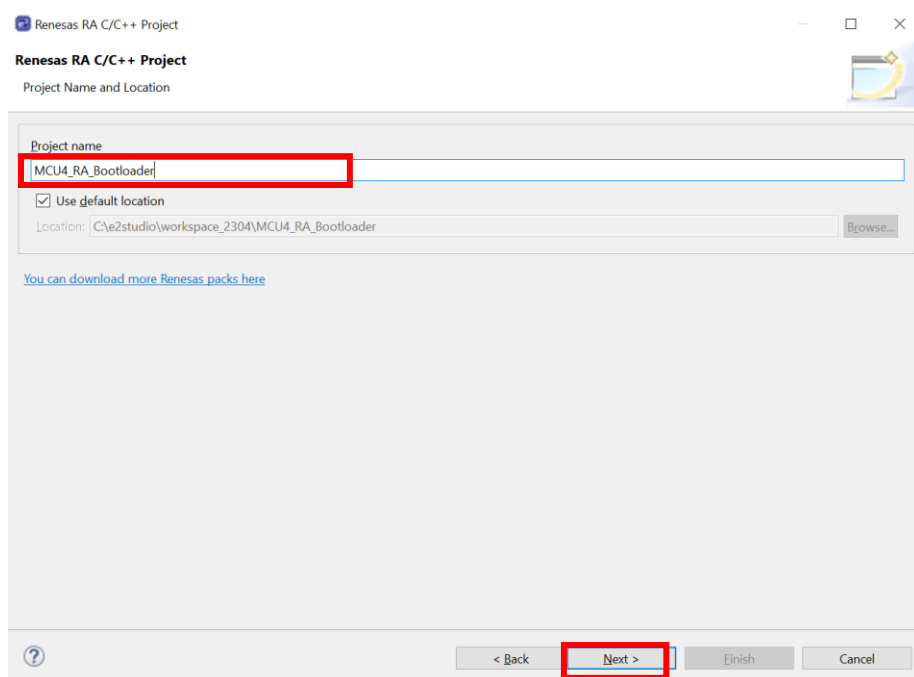
1.1

Launch e2studio and start to establish a new C/C++ Project. Click **File > New > Renesas C/C++ Project > Renesas RA** and then select **C/C++ > Renesas RA C/C++ Project** and press **Next>**.



1.2

Name the project “MCU4_RA_Bootloader” and press **Next**.



Renesas RA C/C++ Project

Project Name and Location

Project name: MCU4_RA_Bootloader

☒ Use default location

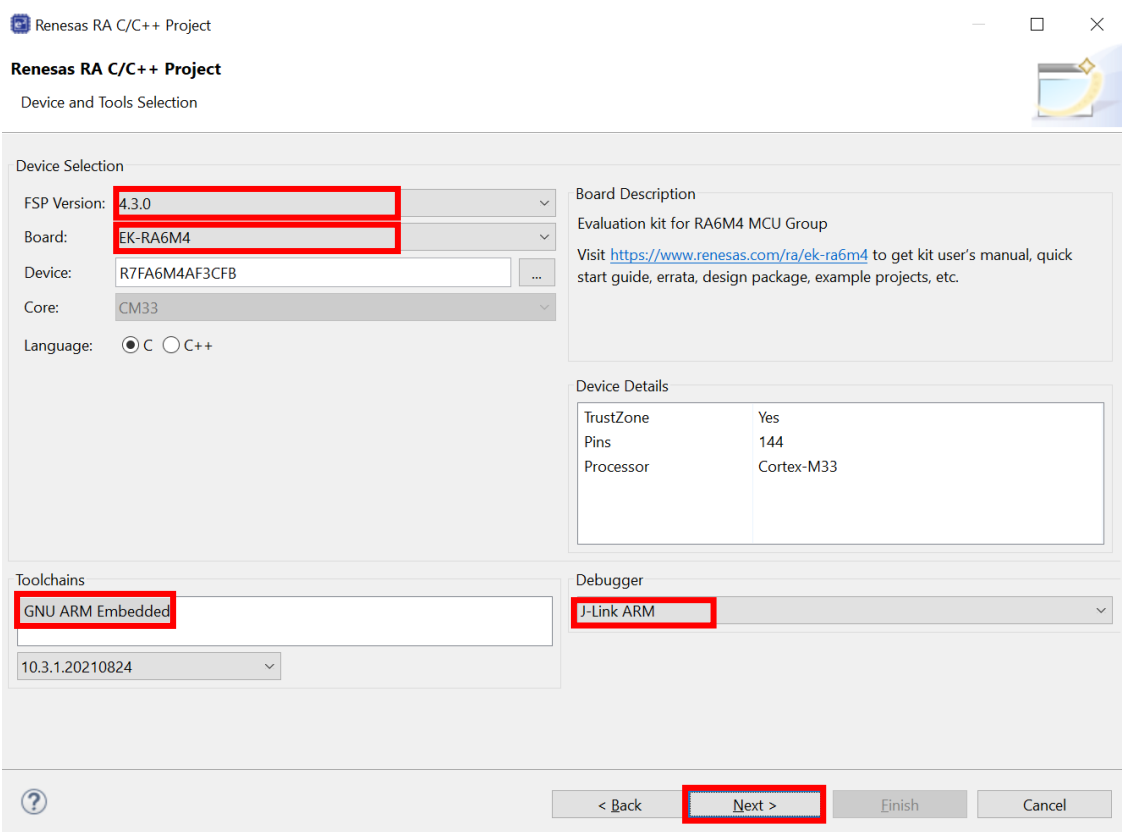
Location: C:\e2studio\workspace_2304\MCU4_RA_Bootloader

You can download more Renesas packs here

< Back Next > Finish Cancel

1.3

Select the **FSP version 4.3.0** and select the board **EK-RA6M4** and verify that the GNU ARM Embedded toolchain and debugger settings are as below and press **Next**.



Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 4.3.0

Board: EK-RA6M4

Device: R7FA6M4AF3CFB

Core: CM33

Language: ☒ C ☐ C++

Board Description

Evaluation kit for RA6M4 MCU Group

Visit <https://www.renesas.com/ra/ek-ra6m4> to get kit user's manual, quick start guide, errata, design package, example projects, etc.

Device Details

TrustZone	Yes
Pins	144
Processor	Cortex-M33

Toolchains

GNU ARM Embedded

10.3.1.20210824

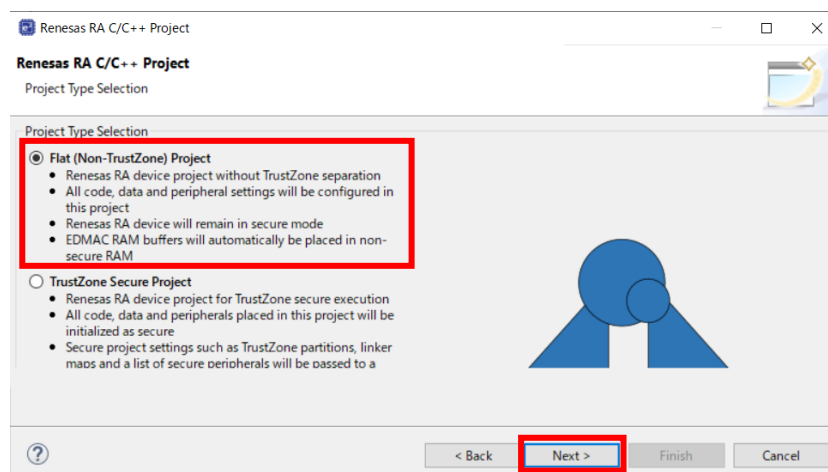
Debugger

J-Link ARM

< Back Next > Finish Cancel

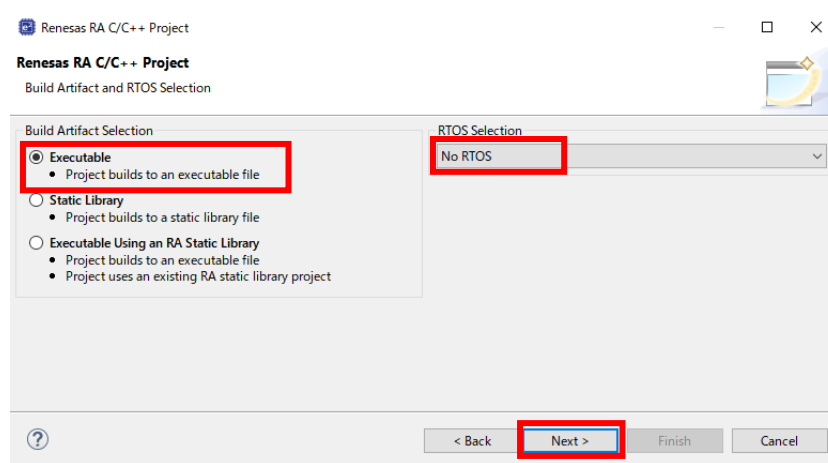
1.4

Select **Flat (Non-TrustZone) Project** as Project Type Selection and then **Next**.



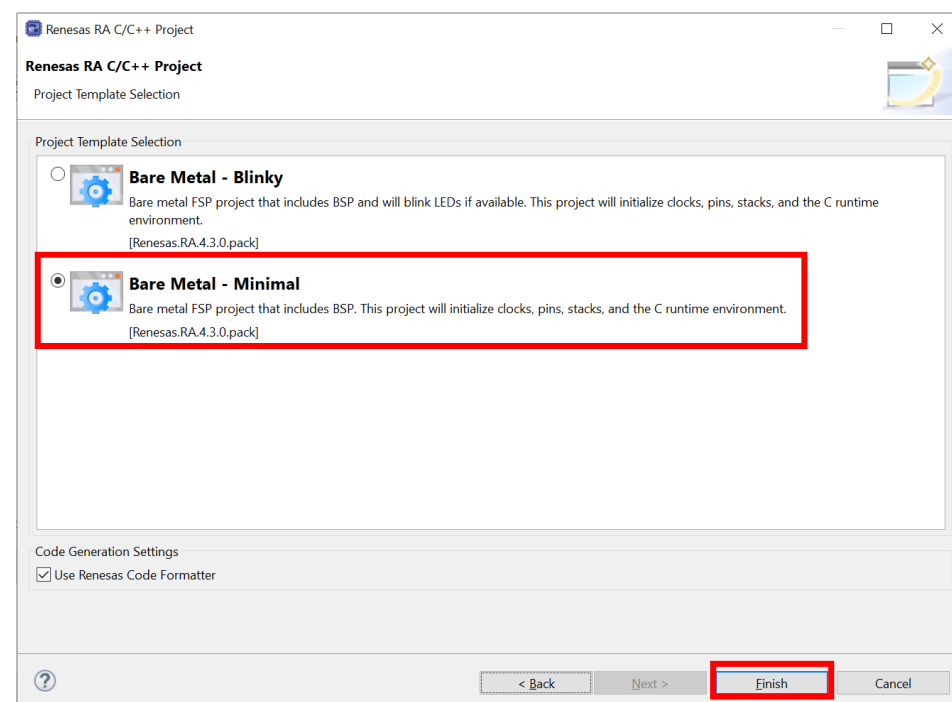
1.5

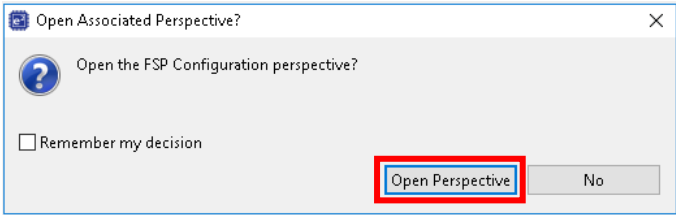
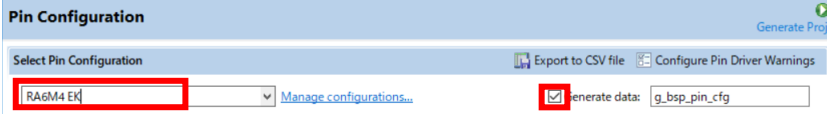
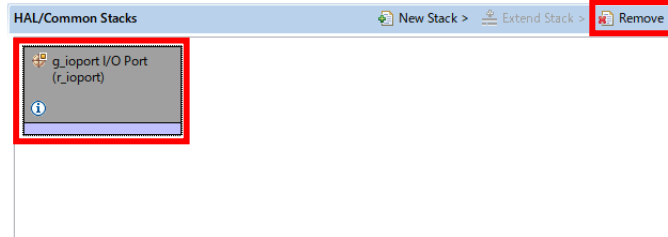
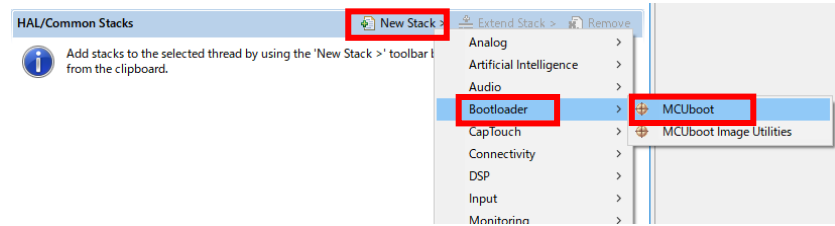
Select **Executable** and **No RTOS** and press **Next**.

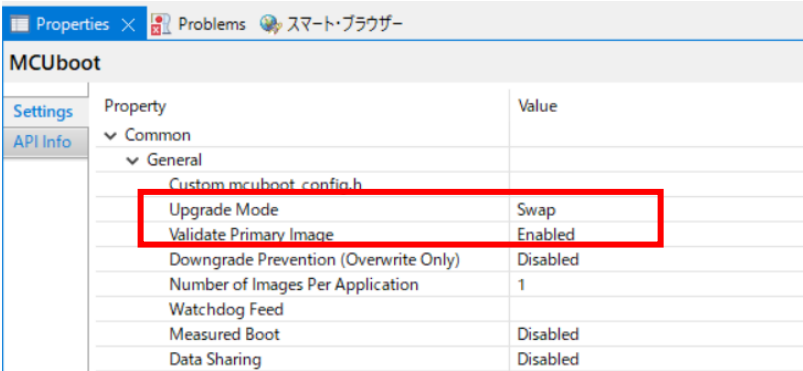
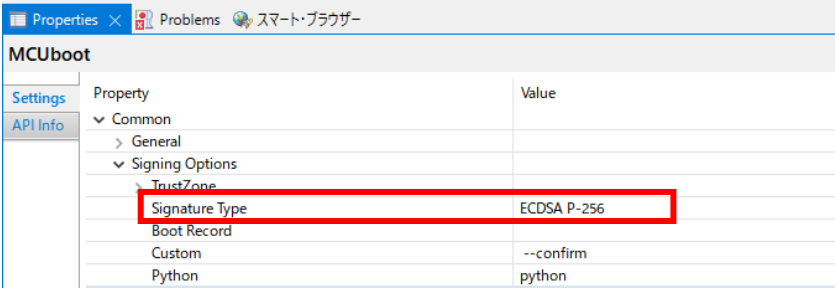
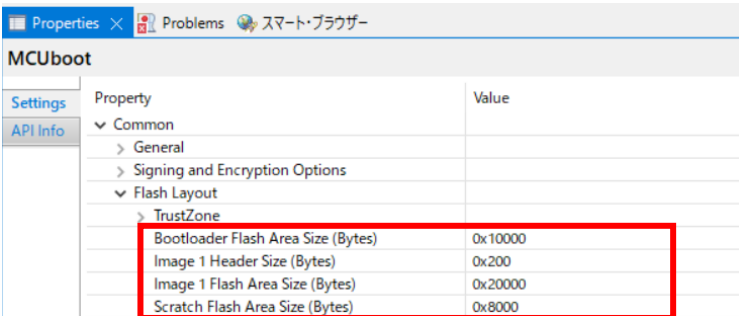


1.6

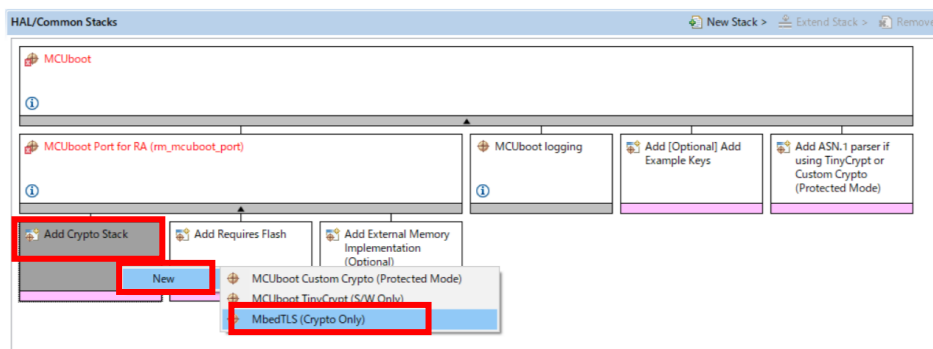
Select **Bare Metal – Minimal** and press **Finish**.



1.7	The project will now be created, and the project configuration will be displayed. When following prompt opens, press Open Perspective. 
1.8	Since this bootloader does not use the extra peripheral and GPIO pins, the pin configuration is not required. Open the Pins tab and uncheck Generate data for RA6M4 EK. 
1.9	Open the Stacks tab and remove I/O Port Driver. 
NOTE	By default, the I/O Port Drive is brought into the project when the project is established. Since the I/O Port Driver is not used in the bootloader project, this stack can be removed.
1.10	Add New Stack > Bootloader > MCUboot. 

1.11	Configure the General properties of MCUboot as below. - Upgrade Mode: Swap - Validate Primary Image: Enabled 
1.12	Configure the Signing Options properties as below. Signature Type: ECDSA P-256 
1.13	Configure the Flash Layout properties as below. - Bootloader Flash Area Size: 0x10000 - Image 1 Header Size: 0x200 - Image 1 Flash Area Size: 0x20000 - Scratch Flash Area Size: 0x8000 
NOTE	The bootloader area size, the scratch area size, the image area size, and the image header size must be selected with attention to the following notes. - The bootloader area size, the scratch area size and the image area size must be multiples of the maximum flash erase block size (0x8000 for RA6M4). - The image header size must be set considering the number of interrupt sources used by the NVIC and the minimum VTOR configurable value (For RA6M4, 0x200 is minimum value in case of if all interrupt sources are enabled) (See KB20425857).

1.14 Click on the **Crypt Stack** block and add **MbedTLS (Crypt Only)** module.

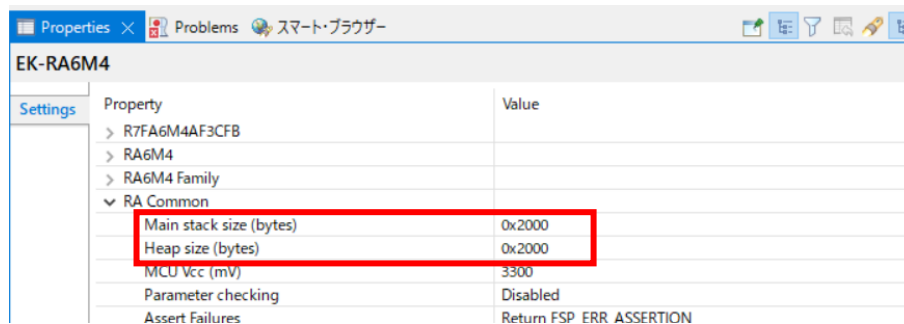


1.15 Configure the MbedTLS module

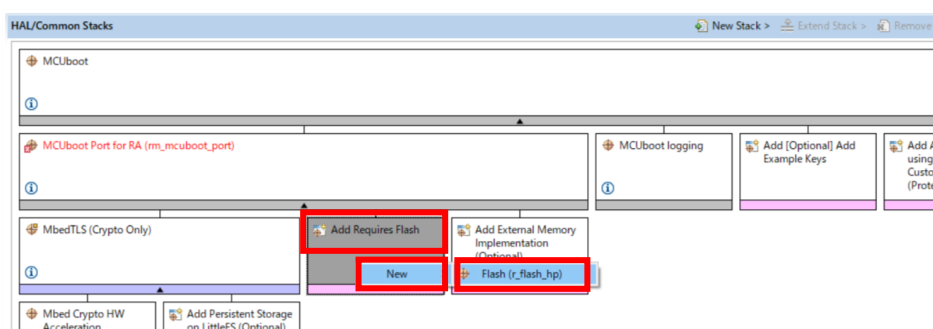
- General > MBEDTLS_THREADING_C: Define → Undefine
- General > MBEDTLS_THREADING_ALT: Define → Undefine
- General > MBEDTLS_MEMORY_BUFFER_ALLOC_C: Undefine → Define

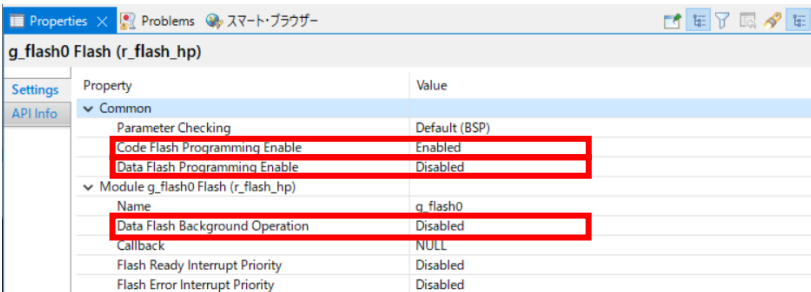
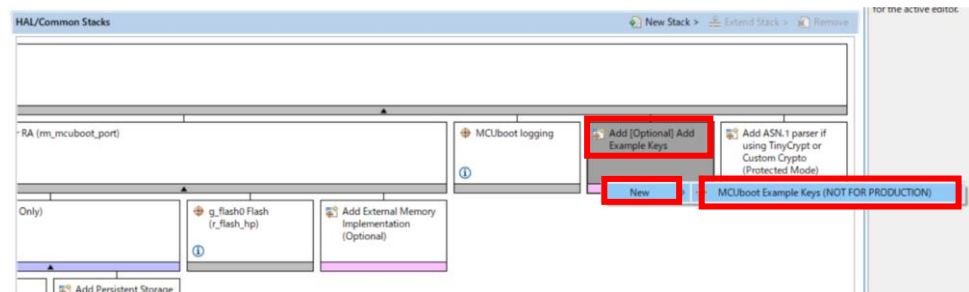
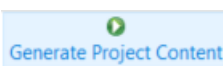
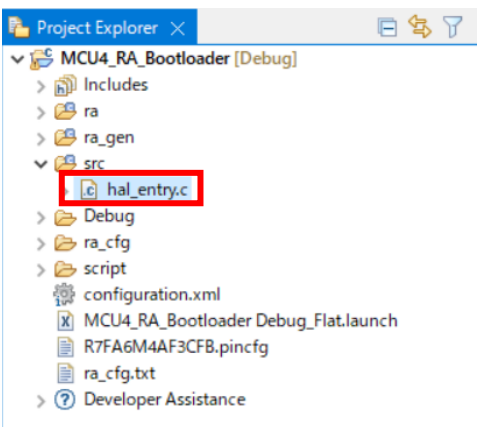
1.16 Open **BSP** tab and change the **Main stack size** and **Heap size**.

- Main stack size: 0x2000
- Heap size: 0x2000



1.17 Now back to **Stacks** tab. Click on the **Requires Flash** block and add **Flash (r_flash_hp)** module.



1.18	Configure the Flash module. As Data Flash Programming and Data Flash Background Operation are not used in the bootloader, select Disabled for these two properties. - Code Flash Programming: Enabled - Data Flash Programming: Disabled - Data Flash Background Operation: Disabled 
1.19	Click on the Add Example Keys block and add MCUboot Example Keys module. 
NOTE	The MCUboot Example Keys are prepared for evaluation use only. DO NOT USED FOR PRODUCTION. For the method of signing keys creation, please refer to signed_images page in MCUboot website.
1.20	Press  to generate the project code.
1.21	In the Project Explorer window, expand the src folder and double click to open the hal_entry.c file to edit it. 

1.22 Expand the **Developer Assistance > HAL/Common > MCUboot > Quick Setup** and then drag and drop the **Call Quick Setup** to line 6 of the `hal_entry.c` file.

- Developer Assistance
 - HAL/Common
 - MCUboot
 - Quick Setup
 - Call Quick Setup**
 - MCUboot Port for RA (rm_mcuboot_port)

1.23 Your file should now look like this:

```

1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  /* Quick setup for MCUboot.
8   *
9   * To update the linker regions of an application to be used with this bootloader, add the *.bld file created during the
10  * build as a File property in e2 studio or RASC under Project > Properties, then C/C++ Build > Build Variables with variable
11  * name BootloaderDataFile.
12  * If QSPI is used, the QSPI flash must be initialized in Extended-SPI mode by calling R_QSPI_Open() before boot_go() is called.
13  */
14  void mcuboot_quick_setup()
15  {
16  #ifdef MCUBOOT_USE_MBED_TLS
17
18  /* Initialize mbedtls. */
19  mbedtls_platform_context ctx = {0};
20  assert(0 == mbedtls_platform_setup(&ctx));
21  #elif (defined(MCUBOOT_USE_TINYCRYPT) && defined(RM_MCUBOOT_PORT_USE_TINYCRYPT_ACCELERATION))
22
23  /* Initialize TinyCrypt port. */
24  assert(FSP_SUCCESS == RM_TINYCRYPT_PORT_Init());
25  #elif (defined(MCUBOOT_USE_USER_DEFINED_CRYPTO_STACK))
26
27  /* Initialize Custom Crypto (Protected Mode) driver. */
28  assert(FSP_SUCCESS == R_SCE_Open(&sce_ctrl, &sce_cfg));
29  #endif
30
31  /* (Optional, not required if --pad is used during signing) To check for updates, call boot_set_pending. */
32  bool update = 0;
33  if (update)
34  {
35      boot_set_pending(0);
36  }
37
38  /* Verify the boot image and get its location. */
39  struct boot_rsp rsp;
40  assert(0 == boot_go(&rsp));
41
42  /* Enter the application. */
43  RM_MCUBOOT_PORT_BootApp(&rsp);
44  }
45
46  /* main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
47  void hal_entry(void)
48  {
49      /* TODO: add your own code here */
50  }
51  */

```

NOTE Using **Properties** view, you can review the code that will be expanded by **Developer Assistance**.

```

/* Quick setup for MCUboot.
 *
 * To update the linker regions of an application to be used with this bootloader, add the *.bld file created during the
 * build as a File property in e2 studio or RASC under Project > Properties, then C/C++ Build > Build Variables with variable
 * name BootloaderDataFile.
 * If QSPI is used, the QSPI flash must be initialized in Extended-SPI mode by calling R_QSPI_Open() before boot_go() is called.
 */
void mcuboot_quick_setup()
{
#ifdef MCUBOOT_USE_MBED_TLS

/* Initialize mbedtls. */
mbedtls_platform_context ctx = {0};
assert(0 == mbedtls_platform_setup(&ctx));
#elif (defined(MCUBOOT_USE_TINYCRYPT) && defined(RM_MCUBOOT_PORT_USE_TINYCRYPT_ACCELERATION))

/* Initialize TinyCrypt port. */
assert(FSP_SUCCESS == RM_TINYCRYPT_PORT_Init());
#elif (defined(MCUBOOT_USE_USER_DEFINED_CRYPTO_STACK))

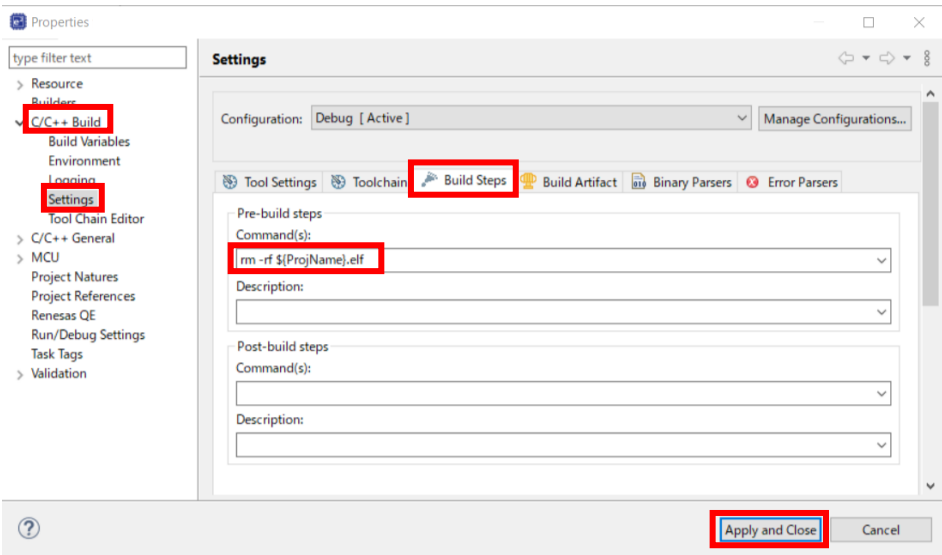
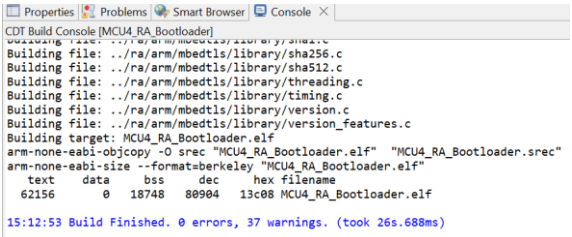
/* Initialize Custom Crypto (Protected Mode) driver. */
assert(FSP_SUCCESS == R_SCE_Open(&sce_ctrl, &sce_cfg));
#endif

/* (Optional, not required if --pad is used during signing) To check for updates, call boot_set_pending. */
bool update = 0;
if (update)
{
    boot_set_pending(0);
}

/* Verify the boot image and get its location. */
struct boot_rsp rsp;
assert(0 == boot_go(&rsp));

/* Enter the application. */
RM_MCUBOOT_PORT_BootApp(&rsp);
}

```

1.24	Add the call to the <code>mcuboot_quick_setup()</code> to the <code>hal_entry()</code> function after TODO comment line at line 52. <pre>void hal_entry(void) { /* TODO: add your own code here */ mcuboot_quick_setup(); ... }</pre>
1.25	Remove all sections of code referencing the I/O Port API. Remove a section of the code in the red boxes in <code>R_BSP_WarmStart()</code> in <code>hal_entry.c</code> file as follows. <pre>78 #endif 79 } 80 81 if (BSP_WARM_START_POST_C == event) 82 { 83 /* C runtime environment and system clocks are setup. */ 84 85 /* Configure pins. */ 86 R_IOPORT_Open (&g_ioport_ctrl, g_ioport.p_cfg); 87 } 88</pre>
1.26	In order to be able to always recompile the project when the linker script is updated, it is recommended add a Pre-build step to always delete the .elf file. Then click Apply and Close. rm -rf \${ProjName}.elf 
NOTE	Starting e2 studio 2023-04, the system command must be based on Linux (Unix)-like command style. E.g., <code>rm</code>, <code>cp</code>.
1.27	Click  to build the MCU4_RA_Bootloader project. Your project should build without any errors. 

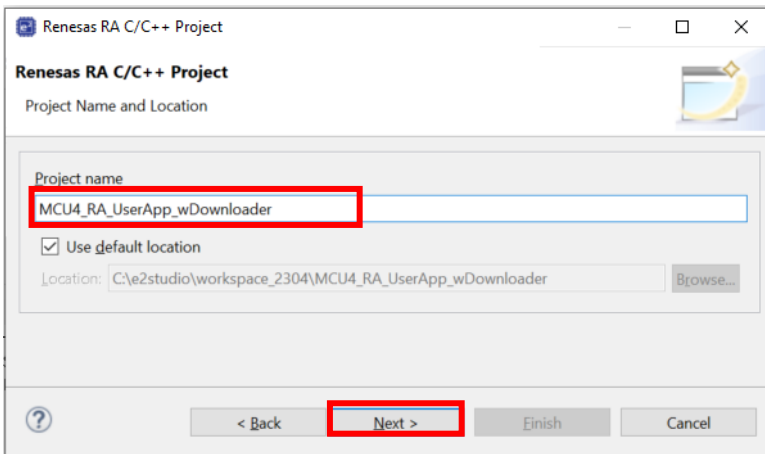
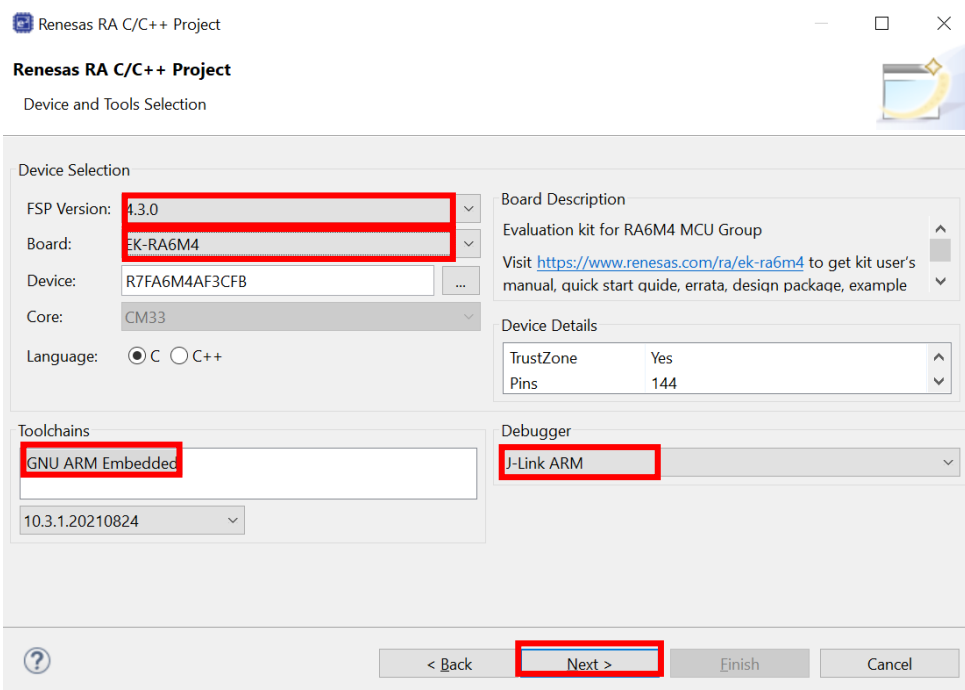
END OF SECTION

2 Creating an Initial User Application Project

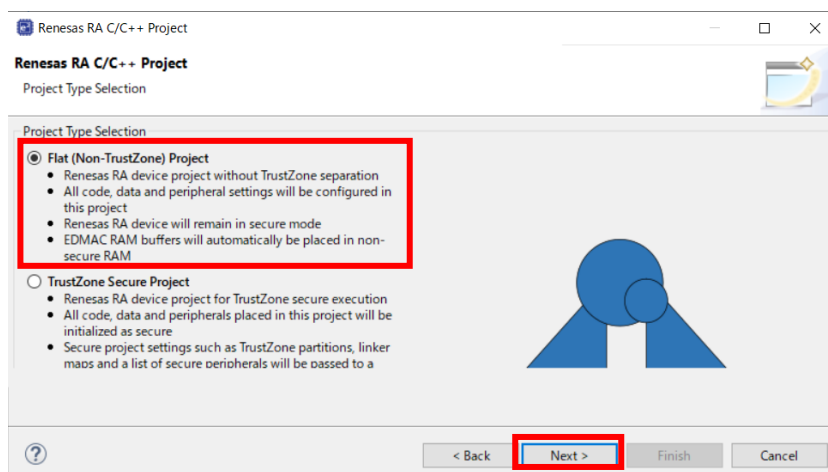
Overview

In this section you will create the application project that will be used for initial application.

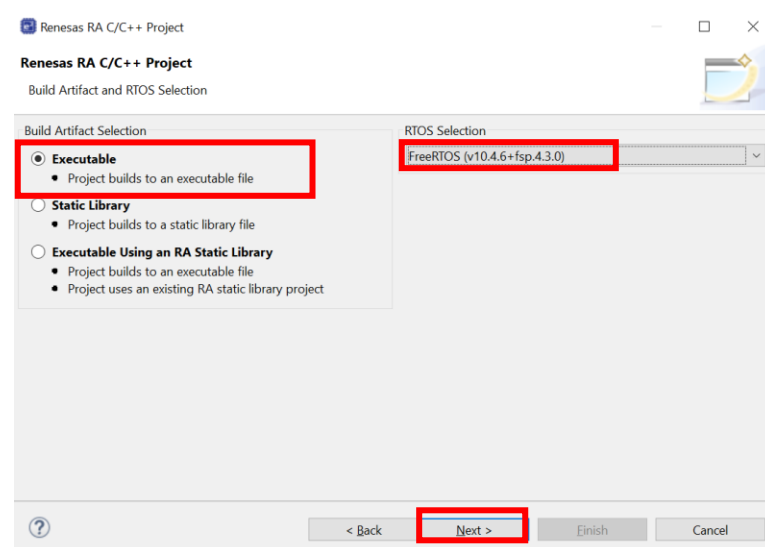
Procedural Steps

2.1	Click File > New > Renesas C/C++ Project > Renesas RA and then select C/C++ > Renesas RA C/C++ Project and press Next .
2.2	Set the project name to “MCU4_RA_UserApp_wDownloader” and press Next. 
2.3	Select the FSP version 4.3.0 and select the used board EK-RA6M4 and verify that the GNU ARM Embedded toolchain and debugger settings are as below and press Next. 

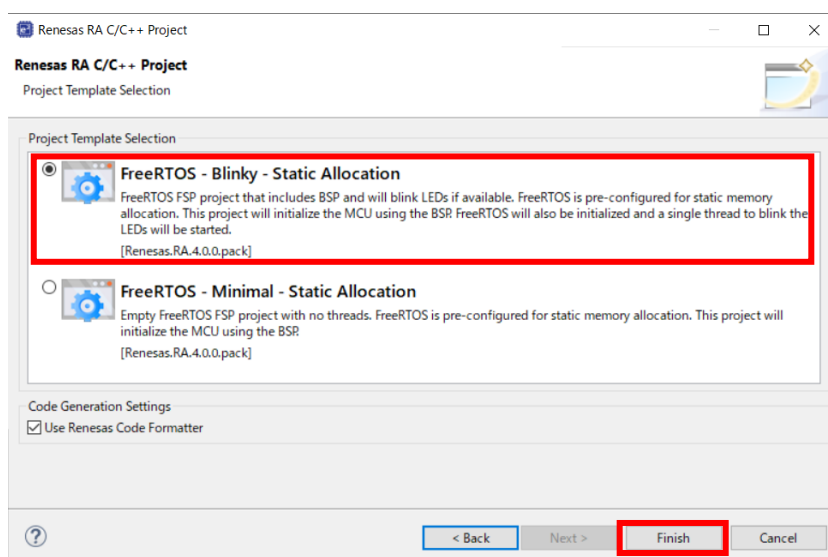
2.4 Select Flat **Flat (Non-TrustZone) Project** as Project Type Selection and then **Next**.



2.5 Select **Executable** and **FreeRTOS(v10.4.6+fspv4.3.0)** and press **Next**.



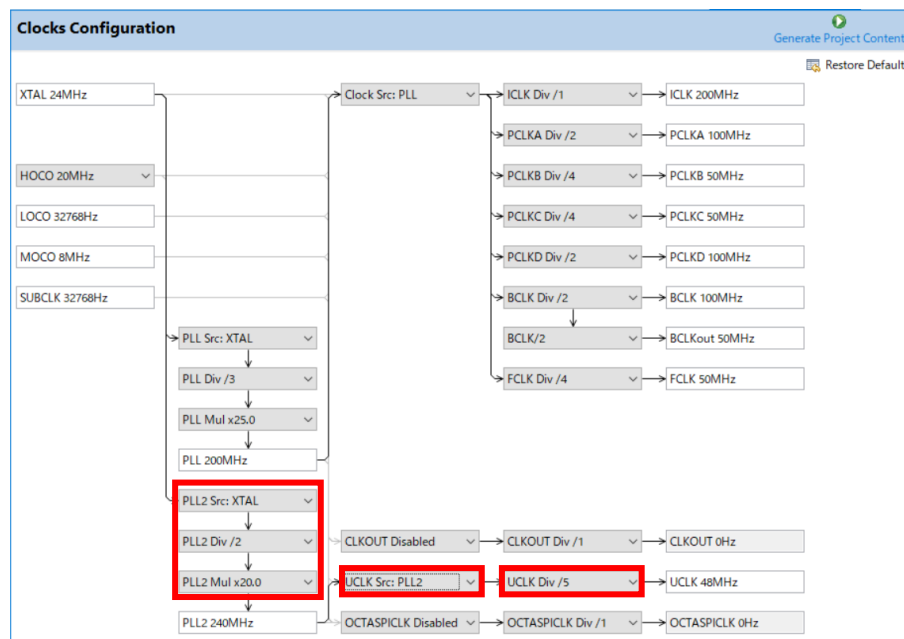
2.6 Select the **FreeRTOS Blinky – Static Allocation** template and press **Finish**.



The project will be created, and the project configuration will be displayed.

2.7

As a clock configuration, enable PLL2 and UCLK with following settings to generate UCLK:48MHz.



NOTE

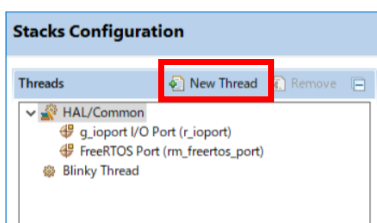
To use USB Device class interface, the USB_VBUS must be enabled in Pins configuration. But if you have selected **EK-RA6M4** as the board selection, the pin configuration has been configured in **RA6M4 EK** pins preset. Therefore, no needed extra configuration for this lab.

The screenshot shows the 'Pin Configuration' window. The 'Select Pin Configuration' dropdown is set to 'RA6M4 EK'. In the 'Pin Selection' list on the left, 'USB_FS0' is highlighted. The 'Pin Configuration' table on the right shows the following settings:

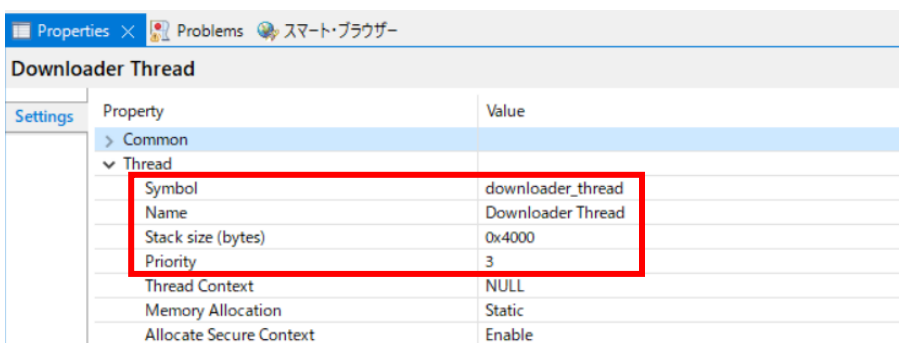
Name	Value	Lock	Link
Operation Mode	Custom		
Input/Output			
USB_DP	USBDP		
USB_DM	USBDM		
USB_OVRCLURB	None		
USB_OVRCLURA	P501		
USB_VBUS	P407		
USB_EXICEN	None		
USB_ID	None		
VCC_USB	VCCUSB		
VSS_USB	VSSUSB		

Below the table, the 'Module name' is 'USB_FS0' and the 'Usage' is 'The below signal list may contain dedicated pins that cannot be configured and are show'.

2.8 Add New Thread “Downloader thread” and configure the thread properties as follows.



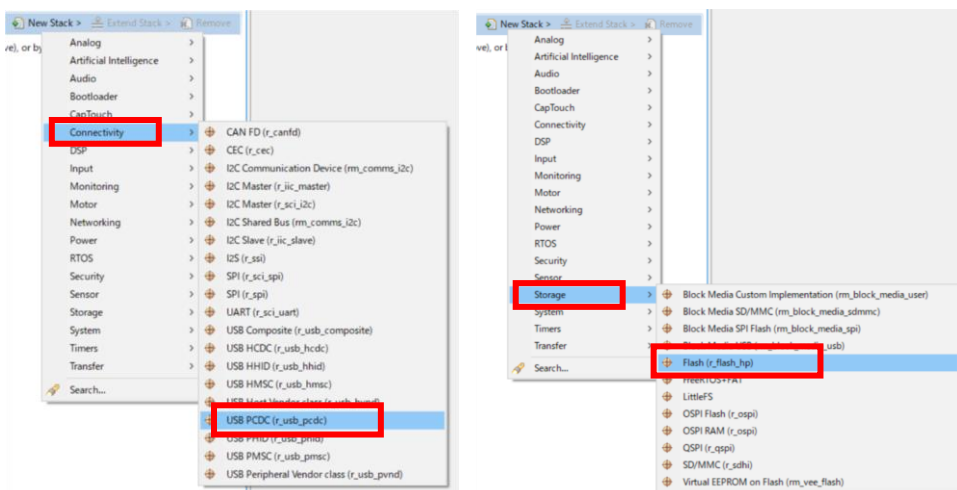
- Thread > Symbol: downloader_thread
- Thread > Name: Downloader Thread
- Thread > Stack size: 0x4000
- Thread > Priority: 3



Also, to avoid the errors related thread name, increase the maximum task name length.

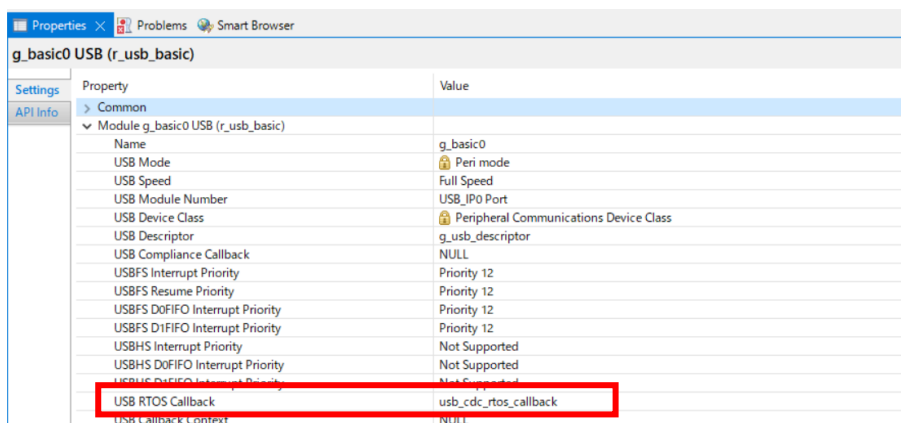
- Common > General > Max Task Name Len: 24

2.9 Add USB PCDC (r_usb_pcdc) module and Flash (r_flash_hp) module on Downloader Thread Stack.



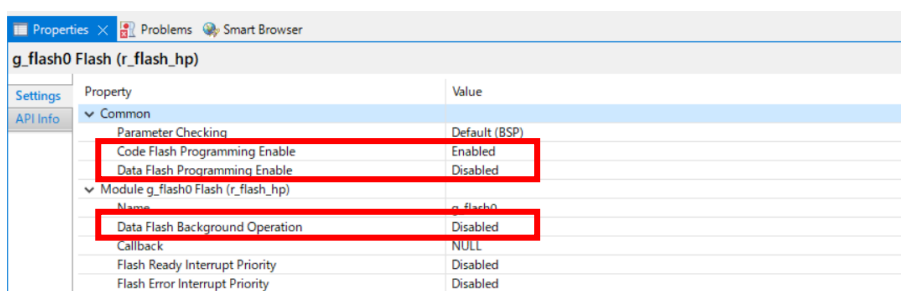
2.10 Configure the USB basic module (**r_usb_basic**) in USB PCDC module tree.

- USB RTOS callback: `usb_cdc_rtos_callback`

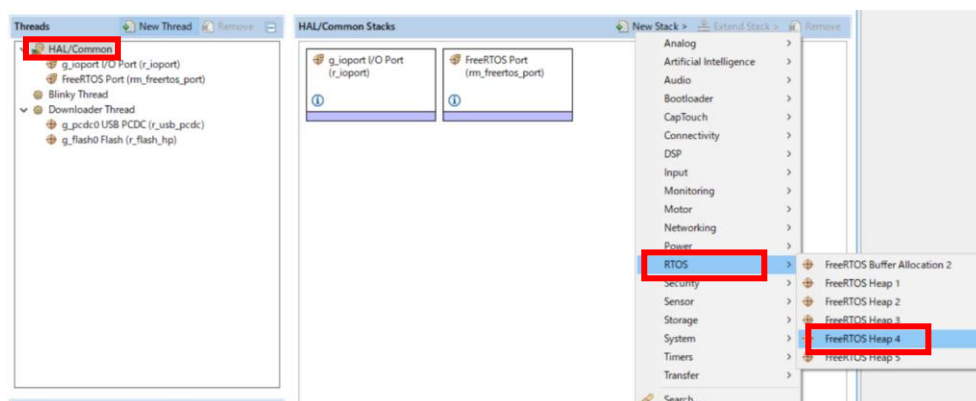


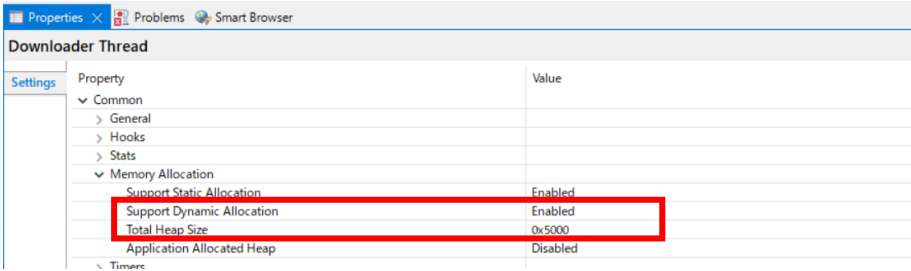
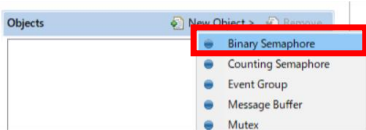
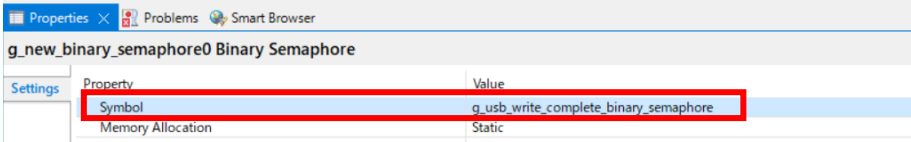
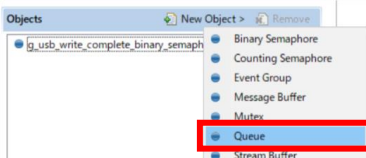
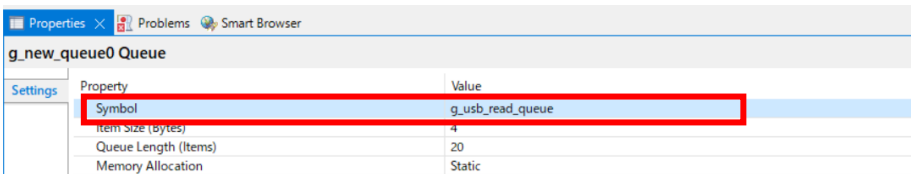
2.11 Configure the **Flash** module.

- Code Flash Programming: Enabled
- Data Flash Programming: Disabled
- Data Flash Background Operation: Disabled



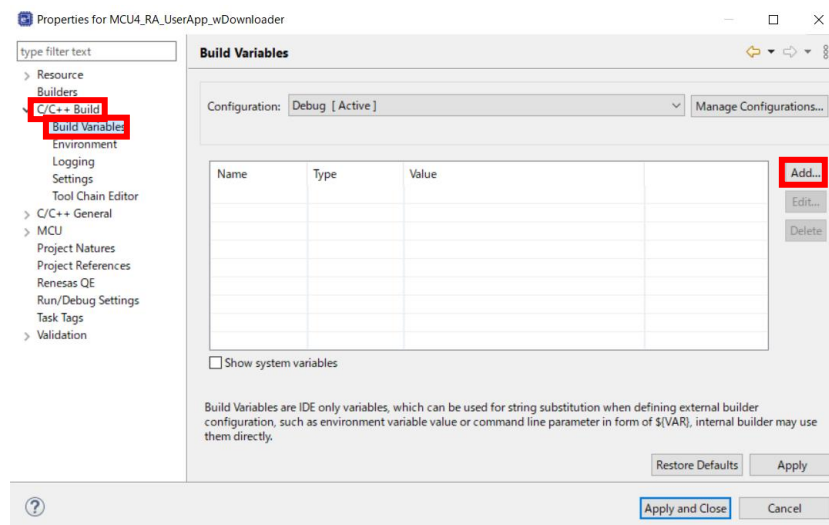
2.12 Add **FreeRTOS Heap4** module onto HAL/Common Thread.



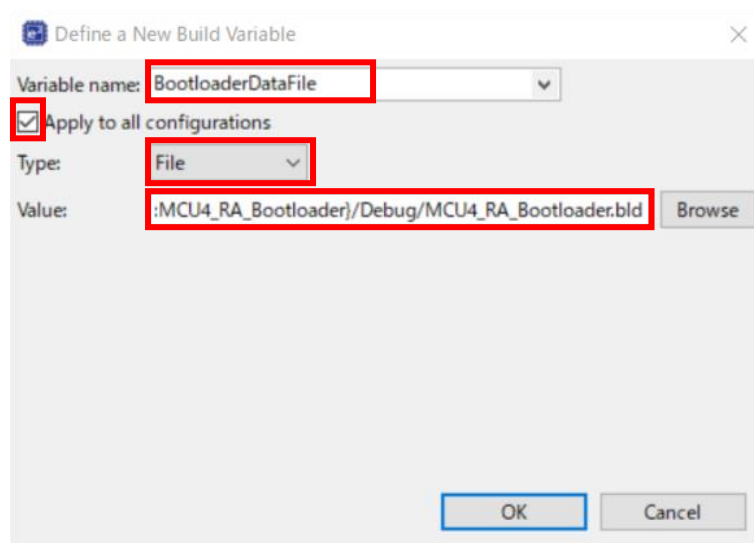
2.13	Enable Dynamic memory allocation to use Heap 4 module. Select any thread and configure as follows: • Common > Memory Allocation > Support Dynamic Allocation: Enabled • Common > Memory Allocation > Total Heap Size: 0x5000 
2.14	Add a Binary Semaphore for USB read complete notification.  And configure the Symbol name to “g_usb_write_complete_binary_semaphore”. 
2.15	Add a Queue for USB write complete notification.  And configure the Symbol name to “g_usb_read_queue”. 
2.16	Click the  button to generate the project code.

2.17

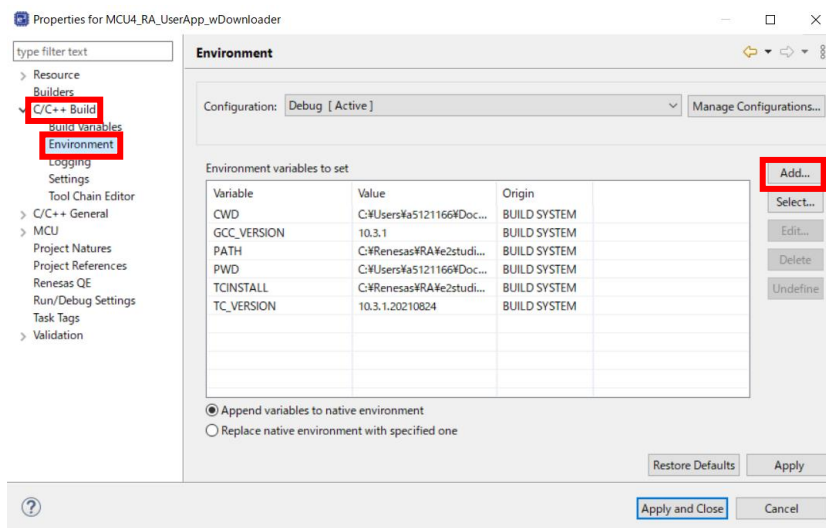
We will now alter the project properties to have it use the bootloader. Right click on the **MCU4_RA_UserApp_wDownloader** folder in the **Project Explorer** window and select **Properties**. Select the **C/C++ Build > Build Variables** and press **Add...**



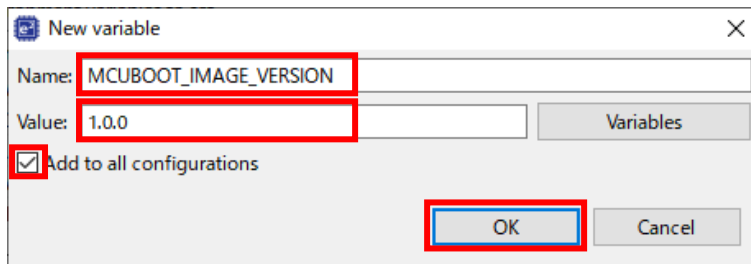
Set the **Variable name** to **"BootloaderDataFile"** and check the **Apply to all configurations** box. Change the **Type** to **File** and enter **"\${workspace_loc:MCU4_RA_Bootloader}/Debug/MCU4_RA_Bootloader.bld"** for the **value**. Click **OK** to save the changes.



2.18 Set the version number of the boot application. Select the **C/C++ Build>Environment** and press **Add....**

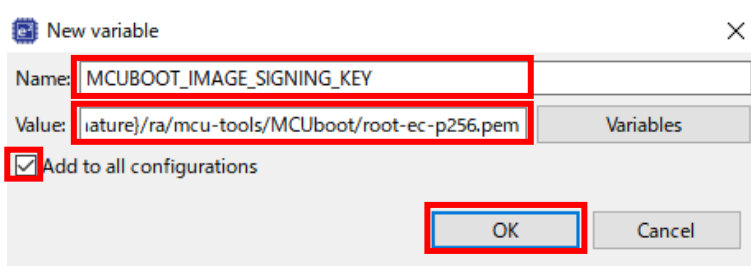


Set the **Name** to “**MCUBOOT_IMAGE_VERSION**” and check the **Apply to all configurations** box. Enter “**1.0.0**” for the **Value**. Click **OK** to save the changes.



2.19 Define the link to the image signing keys. In this section, we are use the **MCUboot example signing keys**, so browse to its file. Select the **C/C++ Build>Environment** and press **Add....**

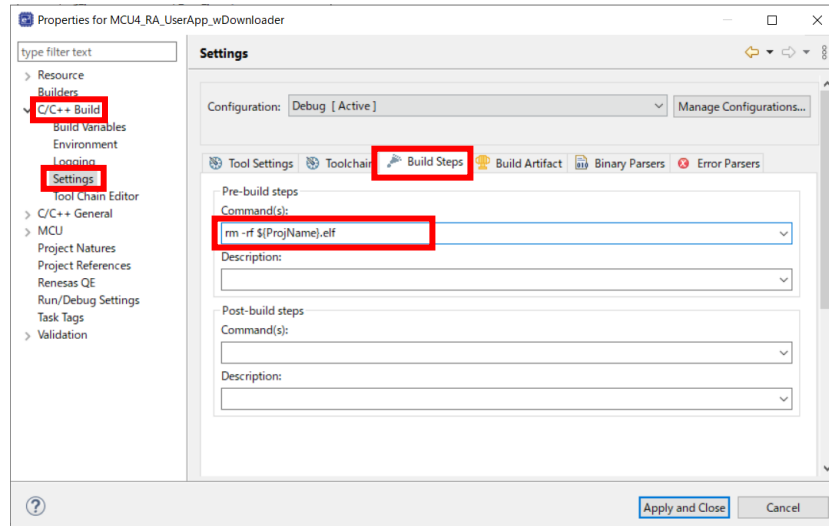
Set **Name** to “**MCUBOOT_IMAGE_SIGNING_KEY**” and click the **Apply to all configuration** box. Enter “**\${workspace_loc:MCU4_RA_Bootloader}/ra/mcu-tools/MCUboot/root-ec-p256.pem**” for the **Value**. Click **OK** to save the changes.



2.20

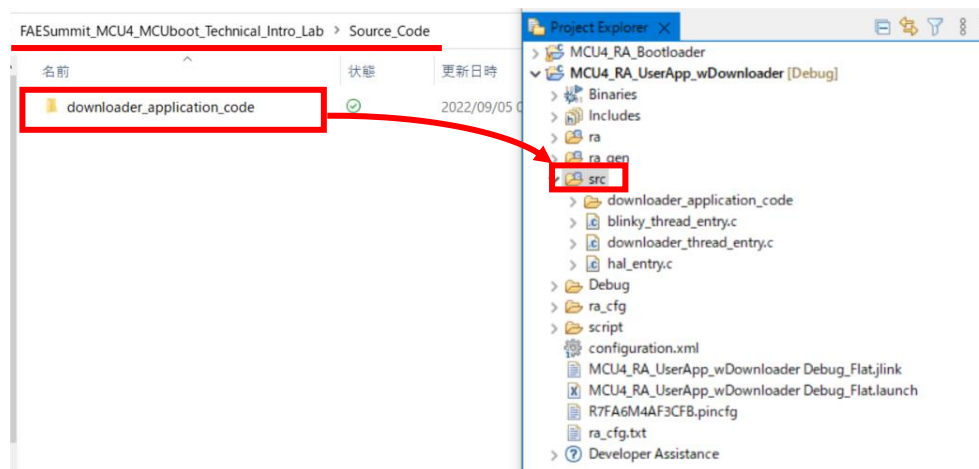
In order to be able to always recompile the project when the **Environment Variables** is updated, it is recommended add a **Pre-build** step to always delete the .elf file. Select the **C/C++ Build > Settings > Build Steps** and enter “**rm -rf \${ProjName}.elf**” in **Pre-build steps > Command(s)**. Then click **Apply and Close**.

Note: If you cannot find out the **Build Steps** tab, please try to switch the display with .

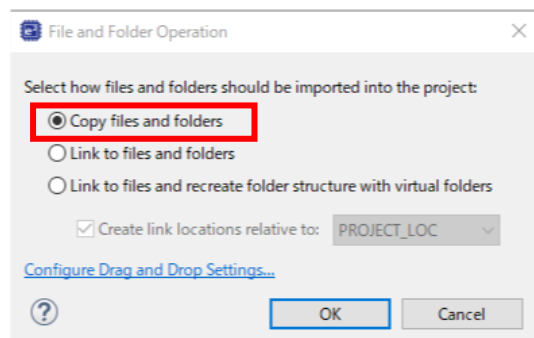


2.21

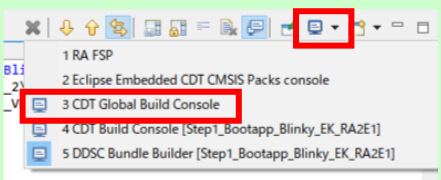
Copy the downloader application code folder “**downloader_application_code**” into **src** folder by drag-and-drop. The source file/folder can be found in project archive zip file.



If the following popup displays, select **Copy files and folders**. Then **OK**.



NOTE	The downloader application code folder consists following codes. - comms.c/.h: Contains USB communication control code - r_usb_pcdc_descriptor.c: Contains USB PCDC descriptor - mcuboot_image_utils.c/.h: Contains MCUboot image utilities code that for this lab app - menu.c/.h: Contains menu and project info displaying code - xmodem.c/.h: Contains operation code for image downloading with XMODEM protocol - common_utils.c/.h: Contains common utilities code - bootloader_image_info.h: Defines image/flash layout information
2.22	Edit the downloader thread entry code in downloader_thread_entry.c as follows. <pre> #include "downloader_thread.h" #include "downloader_application_code/common_utils.h" #include "downloader_application_code/comms.h" #include "downloader_application_code/menu.h" /* Downloader Thread entry function */ void downloader_thread_entry(void *pvParameters) { FSP_PARAMETER_NOT_USED (pvParameters); fsp_err_t fsp_err = FSP_SUCCESS; /* Open the comms driver */ fsp_err = comms_open(); if(FSP_SUCCESS != fsp_err) { ERROR_INDICATER __BKPT(0); } /* Display project info */ project_info(); /* Open Flash driver */ fsp_err = R_FLASH_HP_Open(&g_flash0_ctrl, &g_flash0_cfg); if(FSP_SUCCESS != fsp_err) { ERROR_INDICATER __BKPT(0); } while (1) { /* Display and handle project menu */ menu(); /* Delay 1 system tick (=1ms) */ vTaskDelay(1); } } </pre>

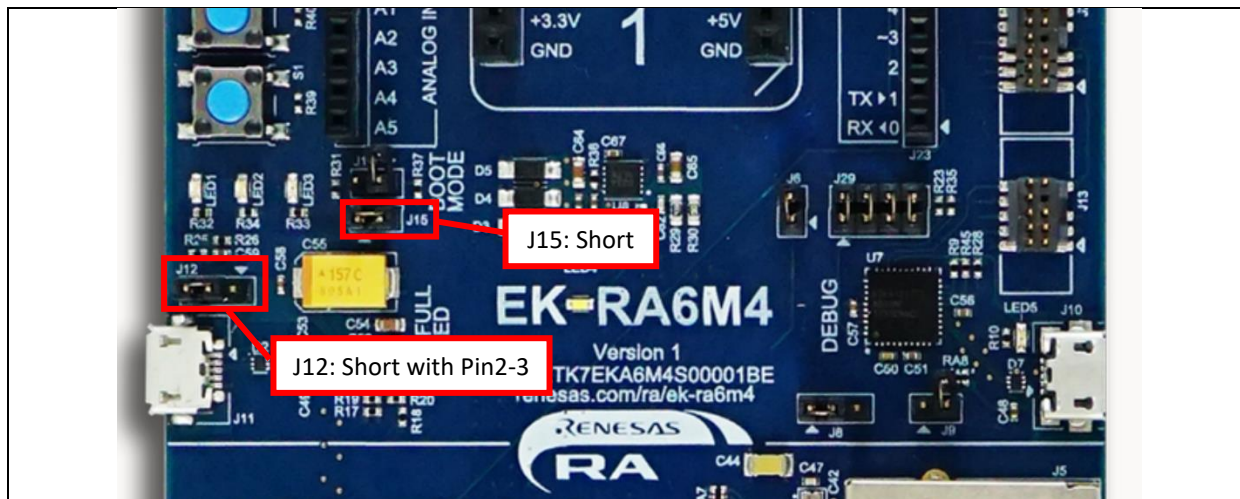
2.23	Edit the blinky thread entry code in blinky_thread_entry.c to enable only LED1 (Blue) for blinky thread. The LED2 (Green) and LED3 (Red) will used in downloader thread. Change the loop end condition from “leds.led_count” to “1” Modified code: <pre> 51 R_BSP_PinAccessEnable(); 52 53 /* Update all board LEDs */ 54 for (uint32_t i = 0; i 1; i++) 55 { 56 /* Get pin to toggle */ 57 uint32_t pin = leds.p_leds[i]; 58 59 /* Write to this pin */ </pre>
2.24	Click  to build the MCU4_RA_UserApp_wDownloader project. The GNU GCC toolchain builds the application and then the image signing tool will start signing the compiled image automatically.  CDT Build Console [MCU4_RA_UserApp_wDownloader] <pre> Building file: ../ra/FreeRTOS/FreeRTOS/Source/portable/MemMang/heap_4.c Building file: ../ra/FreeRTOS/FreeRTOS/Source/event_groups.c Building file: ../ra/FreeRTOS/FreeRTOS/Source/list.c Building file: ../ra/FreeRTOS/FreeRTOS/Source/queue.c Building file: ../ra/FreeRTOS/FreeRTOS/Source/stream_buffer.c Building file: ../ra/FreeRTOS/FreeRTOS/Source/tasks.c Building file: ../ra/FreeRTOS/FreeRTOS/Source/timers.c Building target: MCU4_RA_UserApp_wDownloader.elf arm-none-eabi-objcopy -O srec "MCU4_RA_UserApp_wDownloader.elf" "MCU4_RA_UserApp_wDownloader.srec" arm-none-eabi-size --format=berkeley "MCU4_RA_UserApp_wDownloader.elf" text data bss dec hex filename 44056 0 46008 90064 15fd0 MCU4_RA_UserApp_wDownloader.elf 16:13:29 Build Finished. 0 errors, 0 warnings. (took 7s.814ms) 16:13:29 **** Boot Image Build of configuration Debug for project MCU4_RA_UserApp_wDownloader **** python "C:\\e2studio\\workspace_2304\\MCU4_RA_Bootloader\\ra\\fsp\\src\\rm_mcuboot_port\\rm_mcuboot_port_sign.py" sign --header-size 0x200 --align 128 --max-align 128 --slot-size 0x20000 --max-sectors 4 --confirm --pad-header MCU4_RA_UserApp_wDownloader.elf MCU4_RA_UserApp_wDownloader.bin.signed Packaging application image with version from environment variable MCUBOOT_IMAGE_VERSION: 1.0.0 and key from MCUBOOT_KEY: 1.0.0 --header-size 0x200 --align 128 --max-align 128 --slot-size 0x20000 --max-sectors 4 --confirm --pad-header MCU4_RA_UserApp_wDownloader.elf MCU4_RA_UserApp_wDownloader.bin.signed python imgtool.py sign --key C:\\e2studio\\workspace_2304\\MCU4_RA_Bootloader\\ra\\mcu-tools\\MCUboot\\root-ec-p256.pem --version 1.0.0 --header-size 0x200 --align 128 --max-align 128 --slot-size 0x20000 --max-sectors 4 --confirm --pad-header MCU4_RA_UserApp_wDownloader.elf MCU4_RA_UserApp_wDownloader.bin.signed 16:13:30 Build finished </pre>
NOTE	Full commands of MCUboot image signing for step 2.24 as follows. <pre>python "<e2studio workspace>\\MCU4_RA_Bootloader\\ra\\fsp\\src\\rm_mcuboot_port\\rm_mcuboot_port_sign.py" sign --header-size 0x200 --align 128 --max-align 128 --slot-size 0x20000 --max-sectors 4 --confirm --pad-header MCU4_RA_UserApp_wDownloader.elf MCU4_RA_UserApp_wDownloader.bin.signed</pre>
NOTE	Note on the Console window. The e² studio Console window displays the error messages, logs and information transferred from some tools/software. Normally, when we build the C/C++ code, its message will come in CDT Build Console. However, the image signing tool that used in this RA boot application development will send the message to DDSC Bundle Builder. For more comfortable debugging, we recommended to select CDT Global Build Console. This will display the various type of message globally. 

END OF SECTION

3 Hardware Setup

Before running the project on EK board, we need setup the hardware. In this lab application, we use USBFS Device interface, so the following jumper setting must be set.

- J12 (Enabling of VBUS detection): Short with Pin2-3
- J15 (Enabling of power input from J11): Short



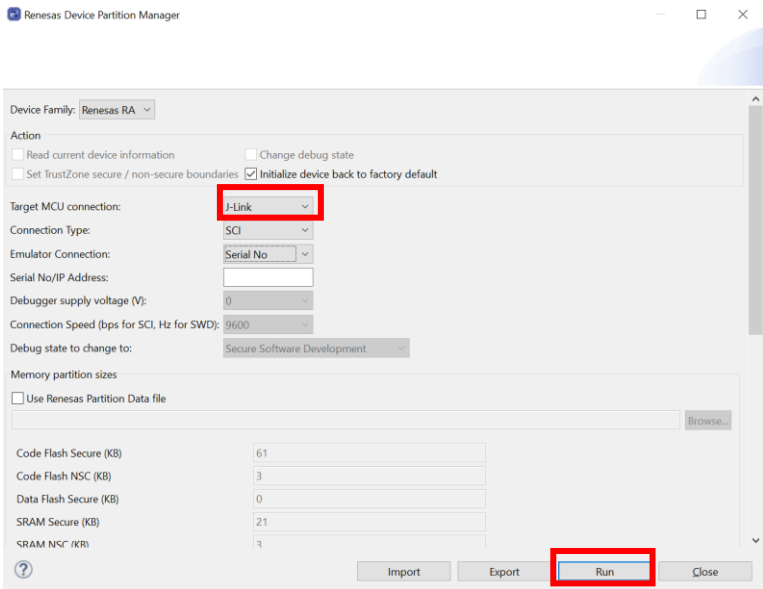
END OF SECTION

4 Download Bootloader Application and Initial User Application project

Overview

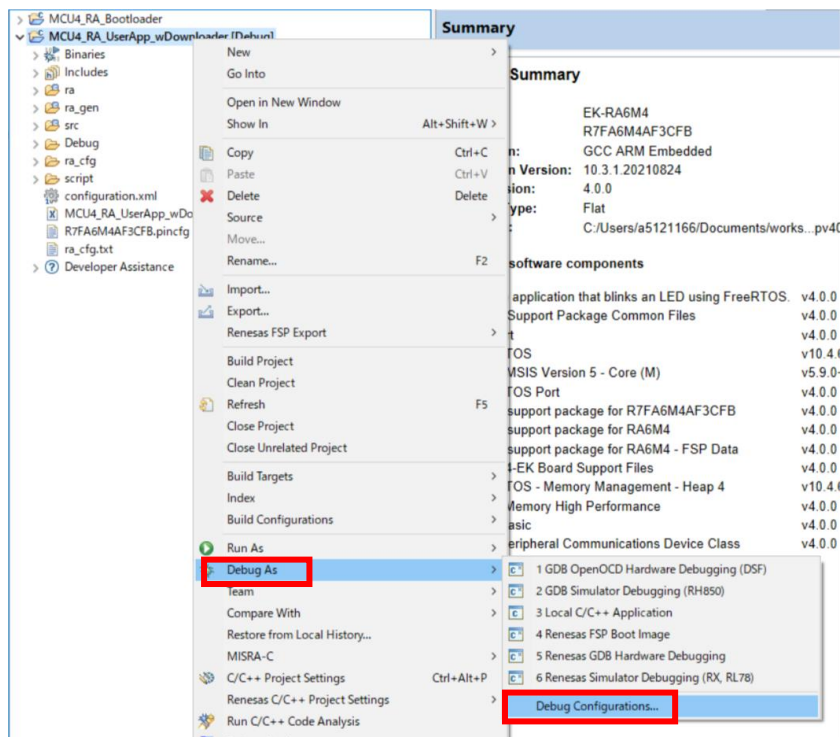
In this section we will modify the debug configurations to allow for the download of bootloader and initial boot application project.

Procedural Steps

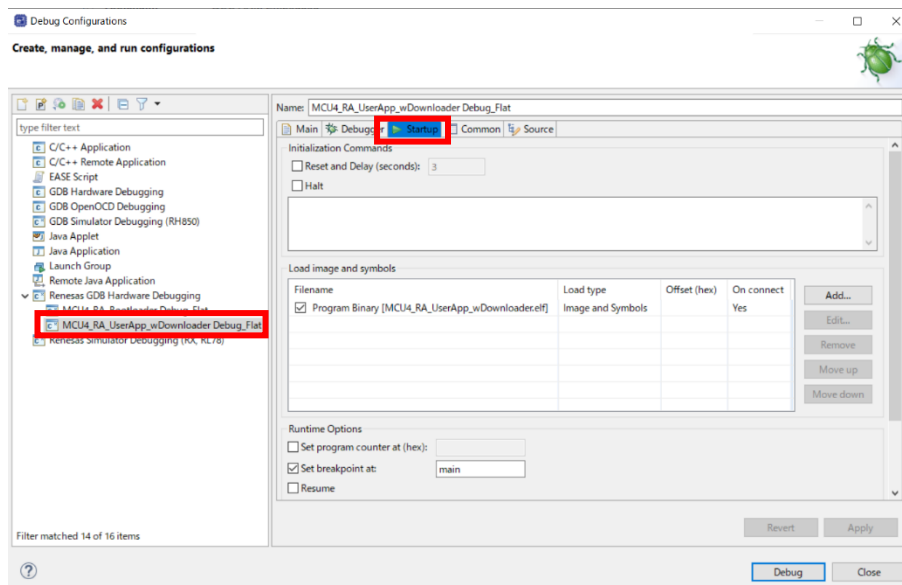
4.1	Connect J10 using a micro-B USB cable to the development PC to provide power and debug connection using the on-board J-Link debugger. 
4.2	At first, erase the flash area using the Renesas Device Partition Manager tool on e² studio. Launch Renesas Device Partition Manager from top bar of e² studio.  Add a check to “Initialize device back to factory default” and select J-Link as Target MCU connection. Then press Run.  If initialization completes successfully, the following message is displayed. <pre> -----SUMMARY OF RESULT----- Connection : SUCCESSFUL! Device initialization : SUCCESSFUL! -----END SUMMARY----- </pre>

NOTE The chip erasing using Renesas Device Partition Management is available only for DLM (Device Life Cycle Management) enabled device (like RA Family - Arm Coretex-M33 device).

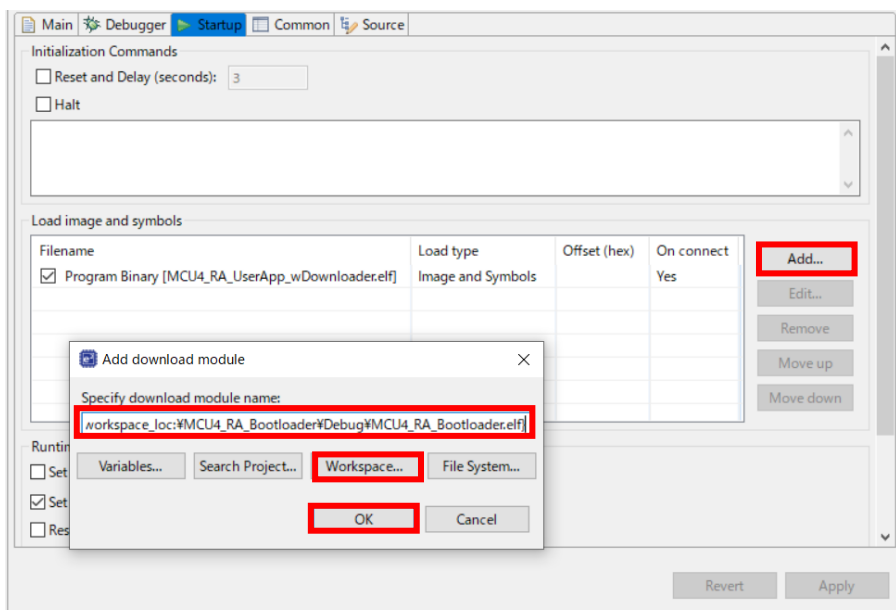
4.3 In the **Project Explorer** window, expand the **MCU4_RA_UserApp_wDownloader** project > **Debug As** and select **Debug Configurations**.



4.4 Make sure the **MCU4_RA_UserApp_wDownloader Debug_Flat** is selected and select the **Startup** tab.



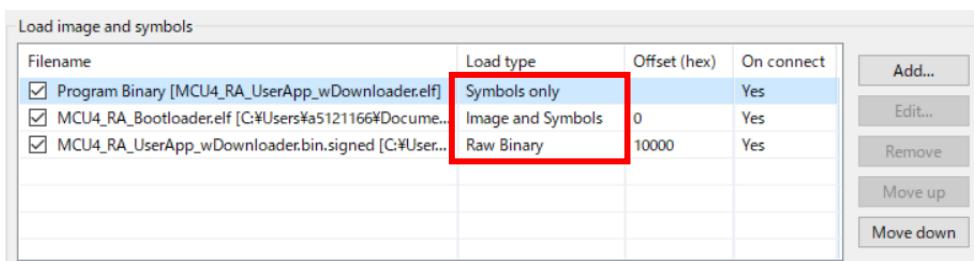
- 4.5 Click **Add...** and then **Workspace**. Navigate to the **MCU4_RA_Bootloader** project and select the **MCU4_RA_Bootloader.elf** file from the **Debug** folder. Press **OK**.



- 4.6 Click **Add...** and then **Workspace**. Navigate to the **MCU4_RA_UserApp_wDownloader** project and select the **MCU4_RA_UserApp_wDownloader.bin.signed** file from the **Debug** folder. Press **OK**.
And change the load type to **Raw Binary** and offset to **"10000"** (Start address of primary slot).

- 4.7 Change the load type of Program Binaries for the **MCU4_RA_UserApp_wDownloader** project to **Symbols only** by clicking on the cell for load type and selecting **Symbols only** from the drop-down menu.

Please verify if your configuration is setup same as following image.

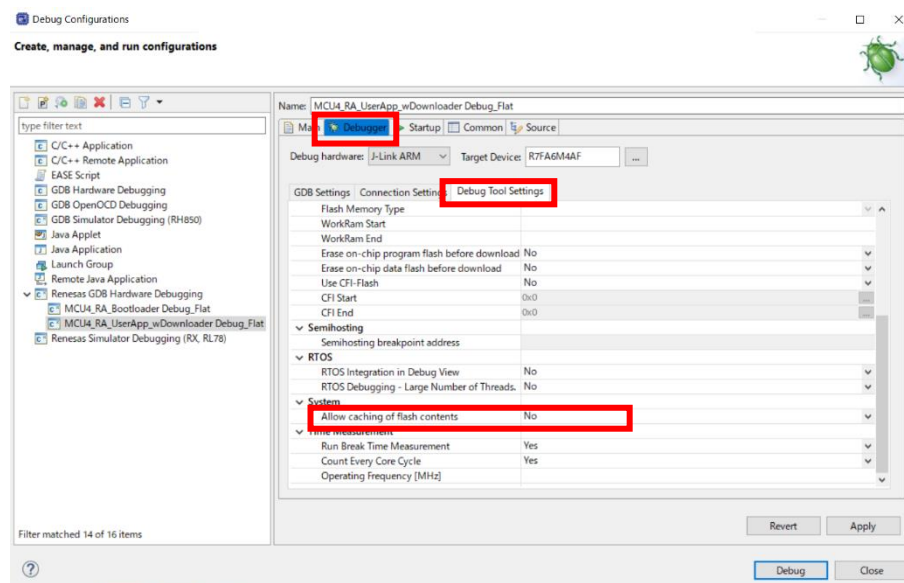


NOTE At step 4.9, during starting the debug session, the following image data will be download to each flash area. The symbol data for debugging will be stored inside of debug probe.

- Bootloader program image
- Initial application program (Signed binary data)
- Bootloader program symbol (For debug)
- Initial application program symbol (For debug)

4.8

Disable Flash content Caching from the **Debugger > Debug Tool Settings > System > Allow caching of flash contents**. Otherwise, the debugging bootloader applications memory window information may show wrong information.

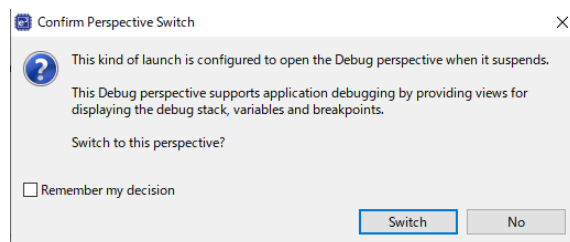


4.9

Press **Debug**

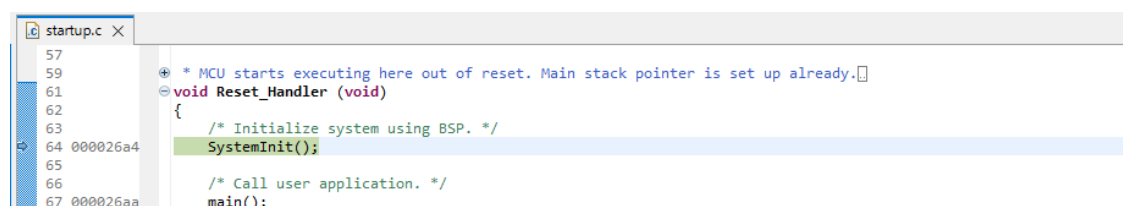
4.10

If the following popup is shown, press **Switch** to switch the perspective that customized for debugging.



4.11

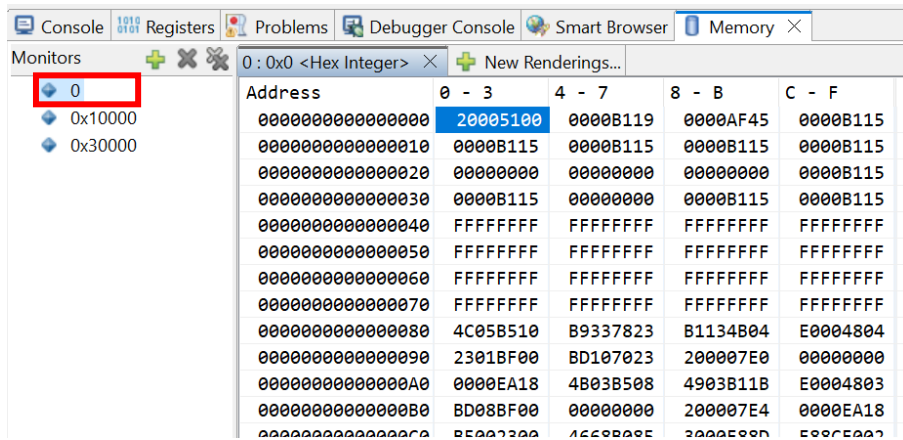
The debugger should hit the reset handler in the bootloader.



4.12

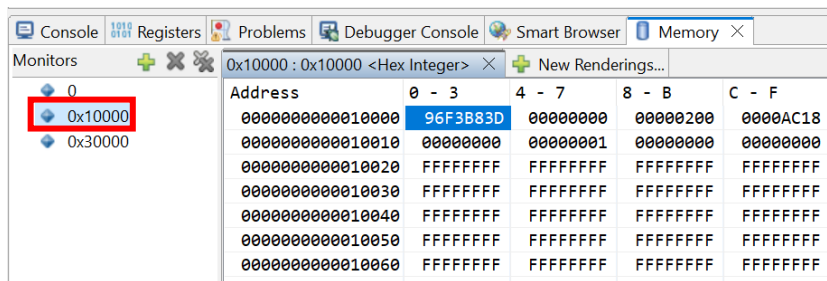
Confirm if the bootloader and initial use application image were written. You can see the current memory status with **Memory** view. The following results should be shown in the below.

- Bootloader area (0x0000_0000 - 0x0000_FFFF): **Bootloader image**



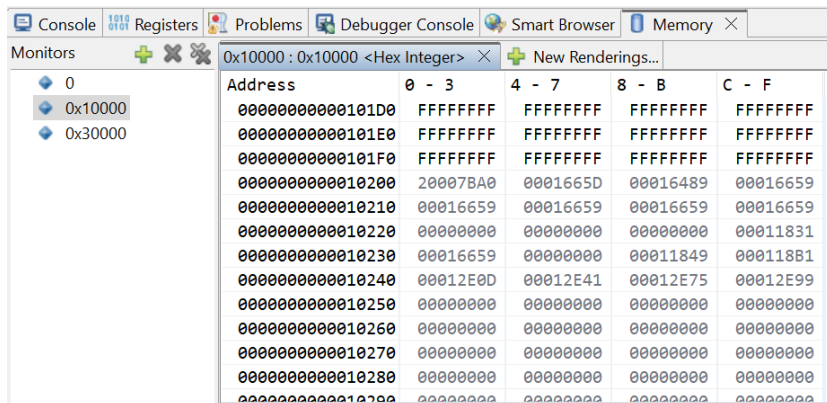
Address	0 - 3	4 - 7	8 - B	C - F
0000000000000000	20005100	0000B119	0000AF45	0000B115
0000000000000010	0000B115	0000B115	0000B115	0000B115
0000000000000020	00000000	00000000	00000000	0000B115
0000000000000030	0000B115	00000000	0000B115	0000B115
0000000000000040	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000000050	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000000060	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000000070	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000000080	4C05B510	B9337823	B1134804	E0004804
0000000000000090	2301BF00	BD107023	200007E0	00000000
00000000000000A0	0000EA18	4B03B508	4903B11B	E0004803
00000000000000B0	BD08BF00	00000000	200007E4	0000EA18
00000000000000C0	00000000	00000000	00000000	00000000

- Primary slot area (0x0001_0000 - 0x0002_FFFF): **Initial user application image**



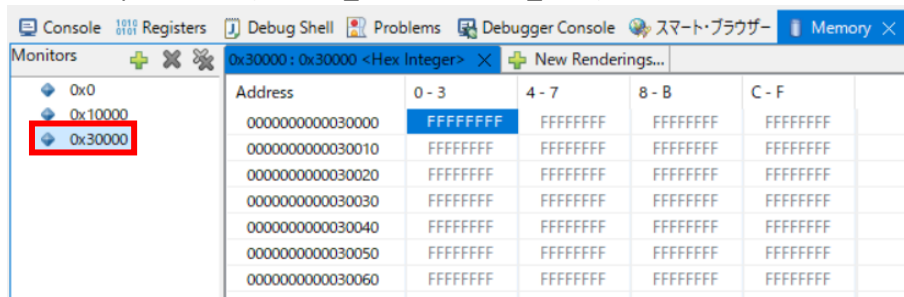
Address	0 - 3	4 - 7	8 - B	C - F
0000000000100000	96F3B83D	00000000	00000200	0000AC18
0000000000100100	00000000	00000001	00000000	00000000
0000000000100200	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000100300	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000100400	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000100500	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000100600	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

...



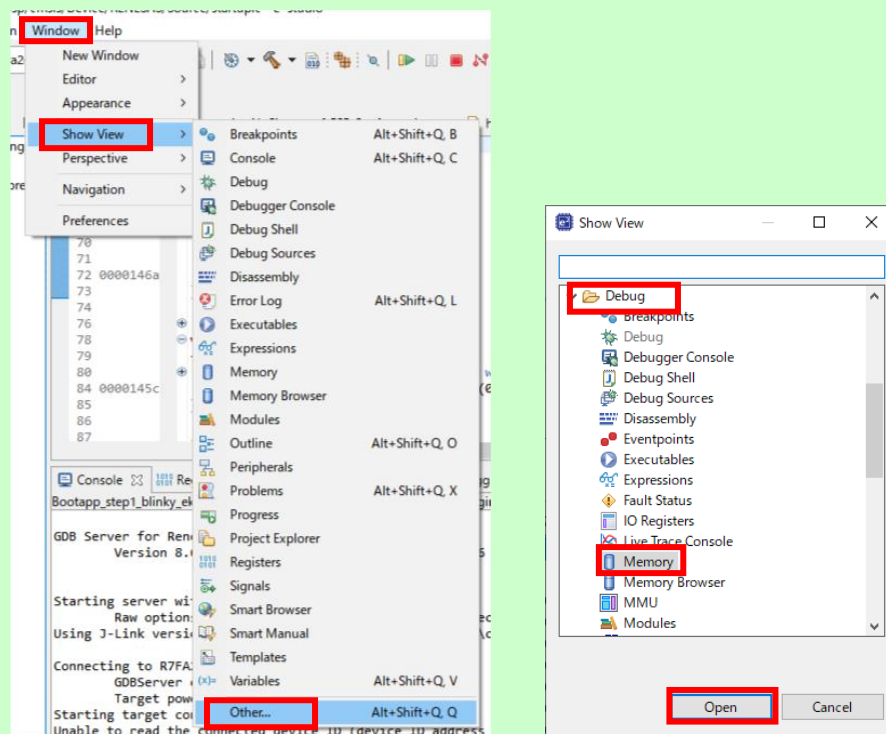
Address	0 - 3	4 - 7	8 - B	C - F
0000000000101D00	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000101E00	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000101F00	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000102000	20007BA0	0001665D	00016489	00016659
0000000000102100	00016659	00016659	00016659	00016659
0000000000102200	00000000	00000000	00000000	00011831
0000000000102300	00016659	00000000	00011849	000118B1
0000000000102400	00012E0D	00012E41	00012E75	00012E99
0000000000102500	00000000	00000000	00000000	00000000
0000000000102600	00000000	00000000	00000000	00000000
0000000000102700	00000000	00000000	00000000	00000000
0000000000102800	00000000	00000000	00000000	00000000

- Secondary slot area (0x0003_0000 - 0x0004_FFFF): **All F**



Address	0 - 3	4 - 7	8 - B	C - F
0000000000300000	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000300100	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000300200	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000300300	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000300400	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000300500	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0000000000300600	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

NOTE If your e2studio does not open the **Memory** view by default, you can open it from e2studio top menu **Windows** tab > **Show View** > **Other...** > **Debug** > **Memory**



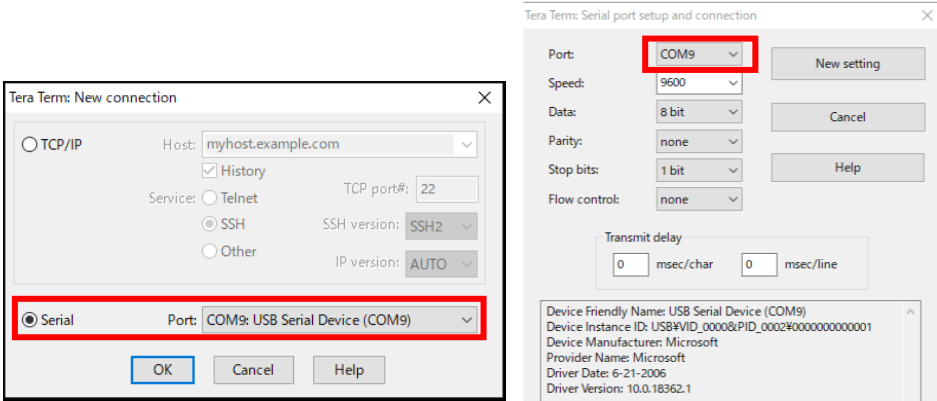
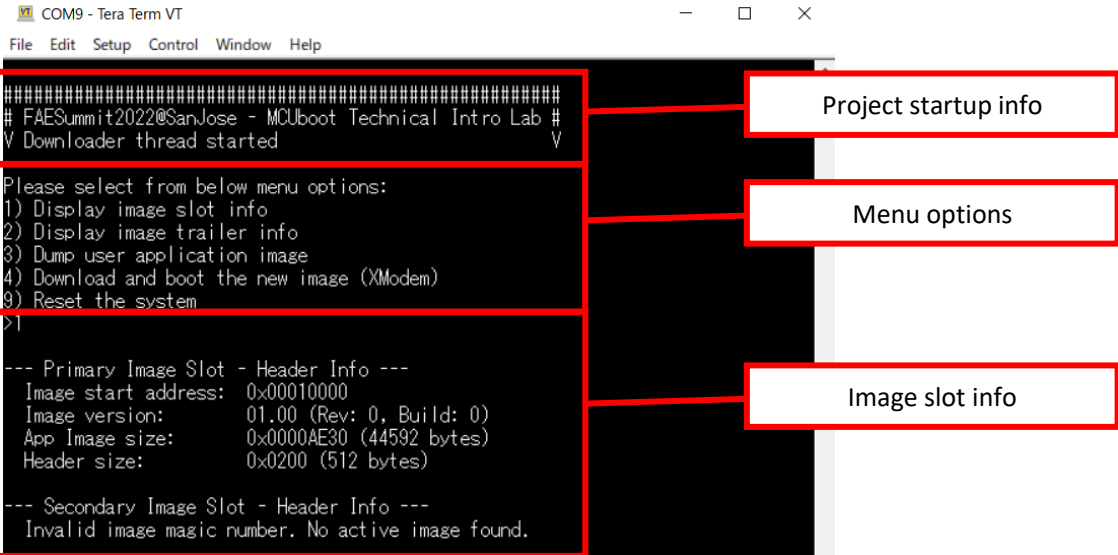
NOTE You can set your specified address to **Monitors** setting from .

4.13 Click  to run the project. If the program paused in main at the hal_entry() call, click  again. The user application in primary slot now starts and the LED1 (Blue) on the board should now be blinking.



4.14 Press the  to pause the program.

Click  to stop the debug session.

4.15	Disconnect micro-B USB cable from J10 on board, and re-connect it to J11. The on-board LED1 (Blue) should blink. From here, the application will be executed with free-run operation, and all application control through Tera Term. 
4.16	Launch Tera Term. Then open virtual COM communication from New connection window, or Setup > Serial port...  Type "1" to Tera Term. If displayed the header information as following example, the COM communication works well. 
NOTE	The COM port that connected to EK-RA6M4 can be found in Device Manager.
NOTE	The serial communication settings are synchronized automatically. So, no need to manual change.

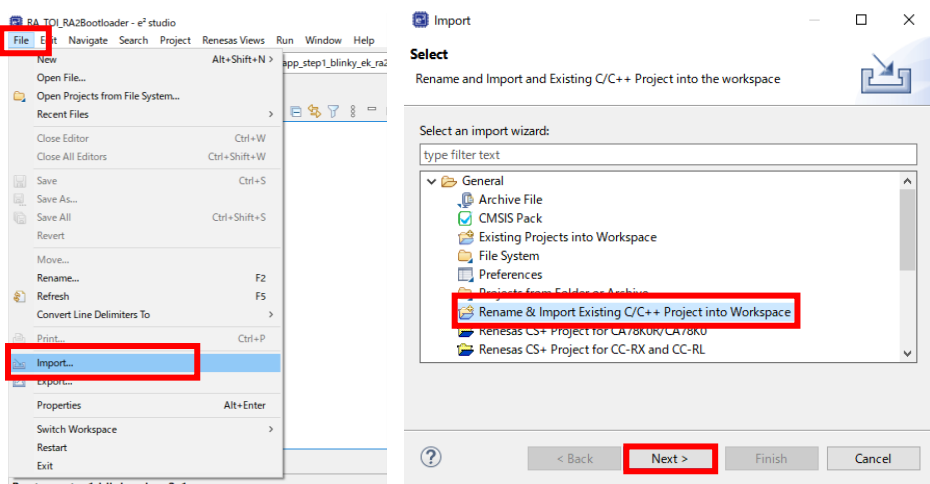
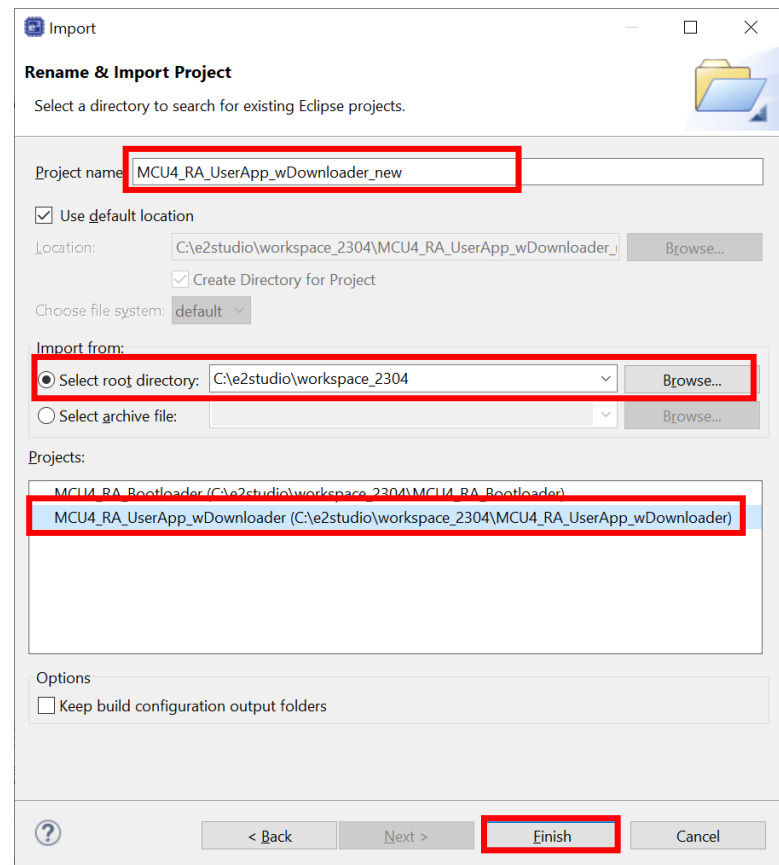
END OF SECTION

5 Upgrading the Application Project

Overview

In this section you will modify the initial user application project created in step 2 and program them into the secondary slot.

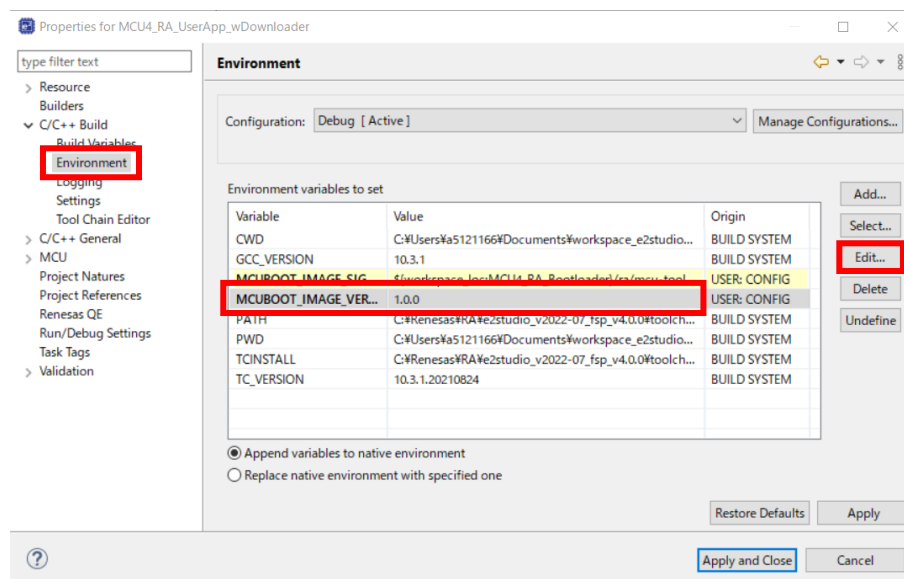
Procedural Steps

5.1	Click File > Import and then select New > Renesas C/C++ Project > Renesas RA and then select General > Rename & Import Existing C/C++ Project into Workspace and press Next>. 
5.2	Name the project “MCU4_RA_UserApp_wDownloader_new” and browse to the base project “MCU4_RA_UserApp_wDownloader” using Browse... button. Then select the project “MCU4_RA_UserApp_wDownloader” in Projects: and then click Finish. 

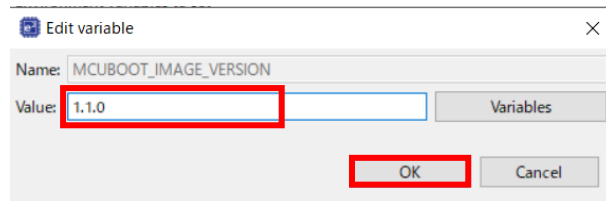
5.3	Go to the bliny_thread_entry.c file in MCU4_RA_UserApp_wDownloader_new project. Currently the MCU4_RA_UserApp_wDownloader_new project blinks LED1 (Blue) with 1s period. Modify the following line of code to do 100ms period blinking. <pre> 73 } 74 } 75 vTaskDelay((configTICK_RATE_HZ)); 76 } 77 </pre> Modified code: <pre> 73 } 74 } 75 vTaskDelay(100); 76 } 77 </pre>
5.4	To user enable to easily recognize the code changes from the top of binary image, update the hard fault exception handler. Open hal_ency.c in user application project MCU4_RA_UserApp_wDownloader_new and add the following code to end of this file. <pre> void HardFault_Handler(void); void HardFault_Handler(void) { __BKPT(1); } </pre>
NOTE	Hard Fault exception handler is one of the Arm Coretex-M CPU system exception. FSP provides this hard fault exception handing code in startup.c with a weak function attribute and also with an alias to default handler. As it uses a weak function attribute, user can overwrite with user-defined one just by defining the code in user source file.

5.5

Update version number of the user application from 1.0.0 to 1.1.0. Select **C/C++ Build > Environment** and **MCUBOOT_IMAGE_VERSION** and the click **Edit...**



Change the **Value** to **"1.1.0"**. Click **OK** to apply the changes.



Then click **Apply and Close**.

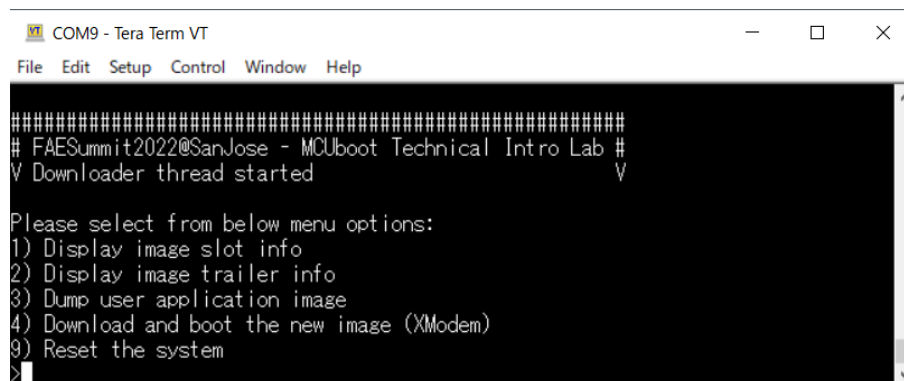
5.6



Click  to build the **MCU4_RA_UserApp_wDownloader_new** project. Your project should build without any errors.

5.7

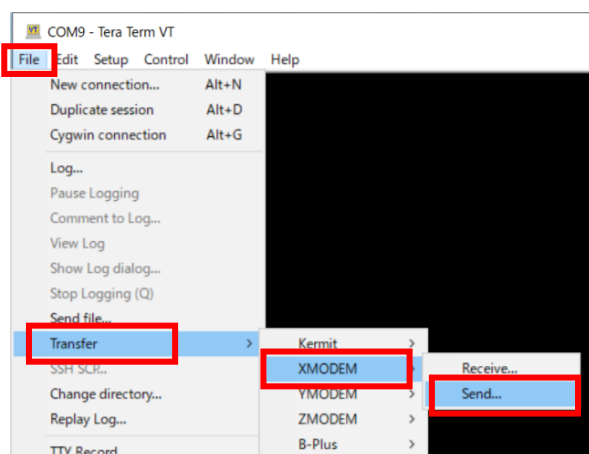
Check the current program. Assuming after re-connect USB cable in 4.15, the LED1(Blue) on the board should now be blinking with 1s period, and the following menu can be seen in Tera Term.



5.8

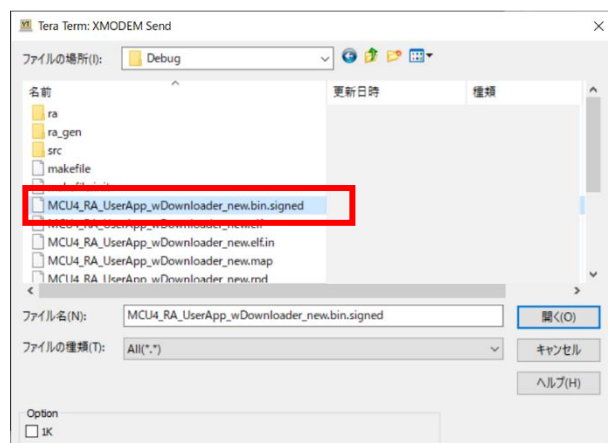
To download new firmware to MCU via XMODEM, type "4" to Tera Term. The downloading operation (Blank check -> Erase secondary slot (if secondary slot is not blanked) -> Send first command) will start.

Then, open the **XMODEM Send** window from **File > Transfer > XMODEM > Send....**



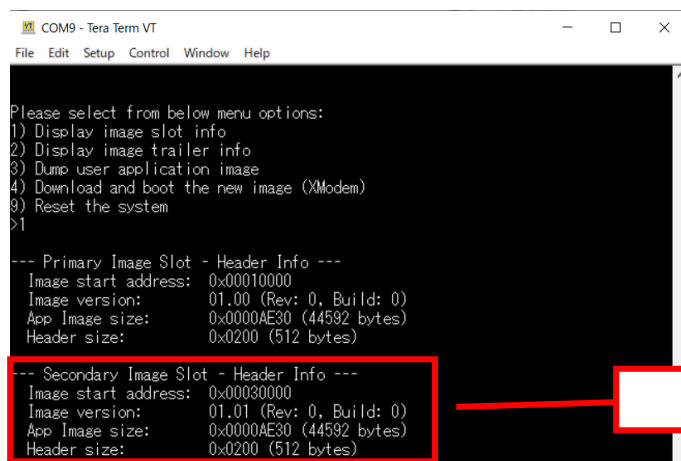
Select **MCU4_RA_UserApp_wDownloader_new.bin.signed** file and press **Open**. The downloading will start.

NOTE: Do NOT add a check to "1K" option.



5.9

Check the status of both image slot. Type “1” to show the image slot information. You can find new firmware in secondary slot with new version number.



```

COM9 - Tera Term VT
File Edit Setup Control Window Help

Please select from below menu options:
1) Display image slot info
2) Display image trailer info
3) Dump user application image
4) Download and boot the new image (XModem)
9) Reset the system
>1

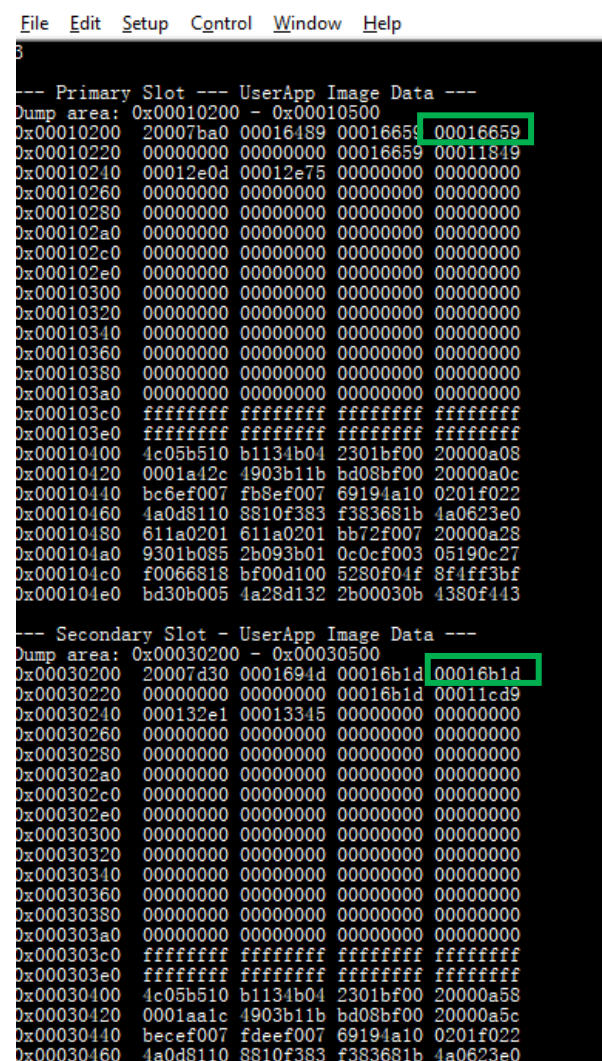
--- Primary Image Slot - Header Info ---
Image start address: 0x00010000
Image version:      01.00 (Rev: 0, Build: 0)
App Image size:     0x0000AE30 (44592 bytes)
Header size:        0x0200 (512 bytes)

--- Secondary Image Slot - Header Info ---
Image start address: 0x00030000
Image version:      01.01 (Rev: 0, Build: 0)
App Image size:     0x0000AE30 (44592 bytes)
Header size:        0x0200 (512 bytes)
  
```

New firmware

Type “3” to show the dump image data.

Please memory the value that shown with green box in below. Values will be swapped after next step.



```

File Edit Setup Control Window Help

3

--- Primary Slot --- UserApp Image Data ---
Dump area: 0x00010200 - 0x00010500
0x00010200  20007ba0 00016189 00016659 00016659
0x00010220  00000000 00000000 00016659 00011819
0x00010240  00012e0d 00012e75 00000000 00000000
0x00010260  00000000 00000000 00000000 00000000
0x00010280  00000000 00000000 00000000 00000000
0x000102a0  00000000 00000000 00000000 00000000
0x000102c0  00000000 00000000 00000000 00000000
0x000102e0  00000000 00000000 00000000 00000000
0x00010300  00000000 00000000 00000000 00000000
0x00010320  00000000 00000000 00000000 00000000
0x00010340  00000000 00000000 00000000 00000000
0x00010360  00000000 00000000 00000000 00000000
0x00010380  00000000 00000000 00000000 00000000
0x000103a0  00000000 00000000 00000000 00000000
0x000103c0  ffffffff ffffffff ffffffff ffffffff
0x000103e0  ffffffff ffffffff ffffffff ffffffff
0x00010400  4c05b510 b1134b04 2301bf00 20000a08
0x00010420  0001a42c 4903b11b bd08bf00 20000a0c
0x00010440  bc6ef007 fb8ef007 69194a10 0201f022
0x00010460  4a0d8110 8810f383 f383681b 4a0623e0
0x00010480  611a0201 611a0201 bb72f007 20000a28
0x000104a0  9301b085 2b093b01 0c0cf003 05190c27
0x000104c0  f0066818 bf00d100 5280f01f 8f4ff3bf
0x000104e0  bd30b005 4a28d132 2b00030b 4380f443

--- Secondary Slot --- UserApp Image Data ---
Dump area: 0x00030200 - 0x00030500
0x00030200  20007d30 0001691d 00016b1d 00016b1d
0x00030220  00000000 00000000 00016b1d 00011cd9
0x00030240  000132e1 00013345 00000000 00000000
0x00030260  00000000 00000000 00000000 00000000
0x00030280  00000000 00000000 00000000 00000000
0x000302a0  00000000 00000000 00000000 00000000
0x000302c0  00000000 00000000 00000000 00000000
0x000302e0  00000000 00000000 00000000 00000000
0x00030300  00000000 00000000 00000000 00000000
0x00030320  00000000 00000000 00000000 00000000
0x00030340  00000000 00000000 00000000 00000000
0x00030360  00000000 00000000 00000000 00000000
0x00030380  00000000 00000000 00000000 00000000
0x000303a0  00000000 00000000 00000000 00000000
0x000303c0  ffffffff ffffffff ffffffff ffffffff
0x000303e0  ffffffff ffffffff ffffffff ffffffff
0x00030400  4c05b510 b1134b04 2301bf00 20000a58
0x00030420  0001aa1c 4903b11b bd08bf00 20000a5c
0x00030440  becef007 fdeef007 69194a10 0201f022
0x00030460  4a0d8110 8810f383 f383681b 4a0623e0
  
```

5.10

Reset the MCU. Type “9” on Tera Term, or press the on-board Reset button.

The MCUboot bootloader swaps the images. If image verification is ok, the user application in primary slot starts and the LED1 (Blue) starts blinking with **high frequency**.

5.11

Check the status of swapped image slot. Type “1” to show the image slot information. You can confirm that the images have been swapped. Assumes new firmware is in primary slot, and old firmware is in secondary slot.

```

COM9 - Tera Term VT
File Edit Setup Control Window Help

Please select from below menu options:
1) Display image slot info
2) Display image trailer info
3) Dump user application image
4) Download and boot the new image (XModem)
5) Reset the system
>1

--- Primary Image Slot - Header Info ---
Image start address: 0x0010000
Image version:      01.01 (Rev: 0, Build: 0)
App Image size:     0x000AE30 (44592 bytes)
Header size:        0x0200 (512 bytes)

--- Secondary Image Slot - Header Info ---
Image start address: 0x00030000
Image version:      01.00 (Rev: 0, Build: 0)
App Image size:     0x000AE30 (44592 bytes)
Header size:        0x0200 (512 bytes)

```

Type “3” to show the dumped image data. As compared with the result on step 5.8, at this point, the primary slot and the secondary slot has been swapped.

```

COM9 - Tera Term VT
File Edit Setup Control Window Help

>3

--- Primary Image Slot - UserApp Image Data ---
Dump area: 0x0010200 - 0x0010500
0x0010200 20007ba0 000164bd 000168d 0001668d
0x0010220 00000000 00000000 0001668d 000164d
0x0010240 00012e51 00012eb5 00000000 00000000
0x0010260 00000000 00000000 00000000 00000000
0x0010280 00000000 00000000 00000000 00000000
0x00102a0 00000000 00000000 00000000 00000000
0x00102c0 00000000 00000000 00000000 00000000
0x00102e0 00000000 00000000 00000000 00000000
0x0010300 00000000 00000000 00000000 00000000
0x0010320 00000000 00000000 00000000 00000000
0x0010340 00000000 00000000 00000000 00000000
0x0010360 00000000 00000000 00000000 00000000
0x0010380 00000000 00000000 00000000 00000000
0x00103a0 00000000 00000000 00000000 00000000
0x00103c0 ffffffff ffffffff ffffffff ffffffff
0x00103e0 ffffffff ffffffff ffffffff ffffffff
0x0010400 4c05b510 b1134b04 2301bf00 20000a50
0x0010420 0001a474 4903b11b bd08bf00 20000a54
0x0010440 bc86f007 fba8f007 23e0f04f 69196008
0x0010460 0201f022 f04fb008 61984a09 0201f022
0x0010480 0201f042 8810f383 bb8af007 20000a6c
0x00104a0 9301b085 2b093b01 0c0cf003 05190c24
0x00104c0 f0066818 bf00d100 5280f04f 8f4ff3bf
0x00104e0 bd30b005 f1046810 fce8f006 a902e7e7

--- Secondary Image Slot - UserApp Image Data ---
Dump area: 0x00030200 - 0x00030500
0x00030200 20007ba0 000164b9 0001689 00016689
0x00030220 00000000 00000000 00016689 000164d
0x00030240 00012e4d 00012eb1 00000000 00000000
0x00030260 00000000 00000000 00000000 00000000
0x00030280 00000000 00000000 00000000 00000000
0x000302a0 00000000 00000000 00000000 00000000
0x000302c0 00000000 00000000 00000000 00000000
0x000302e0 00000000 00000000 00000000 00000000
0x00030300 00000000 00000000 00000000 00000000
0x00030320 00000000 00000000 00000000 00000000
0x00030340 00000000 00000000 00000000 00000000
0x00030360 00000000 00000000 00000000 00000000
0x00030380 00000000 00000000 00000000 00000000
0x000303a0 00000000 00000000 00000000 00000000
0x000303c0 ffffffff ffffffff ffffffff ffffffff
0x000303e0 ffffffff ffffffff ffffffff ffffffff
0x00030400 4c05b510 b1134b04 2301bf00 20000a50
0x00030420 0001a474 4903b11b bd08bf00 20000a54
0x00030440 bc84f007 fba4f007 23e0f04f 69196008
0x00030460 0201f022 f04fb008 61984a09 0201f022
0x00030480 0201f042 8810f383 bb88f007 20000a6c
0x000304a0 9301b085 2b093b01 0c0cf003 05190c24
0x000304c0 f0066818 bf00d100 5280f04f 8f4ff3bf
0x000304e0 bd30b005 f1046810 fce8f006 a902e7e7

```

END OF SECTION

6 Testing the revert mechanism of MCUboot Swap test mode

Overview

In this section, we will test the reverting of MCUboot Swap test mode. In Swap mode, the old firmware image can be stored in secondary slot, and the system can revert to previous firmware using it when the faulty update.

The bootloader checks the image_ok byte stored in primary slot image, and then decides whether to perform a revert. With default MCUboot configuration, the image_ok byte is set as 0x1 (Not revert) at build-time and cannot perform the reverting. To enable the revert function, change the MCUboot configuration to be able to set image_ok at run-time.

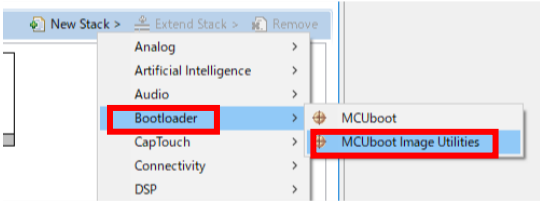
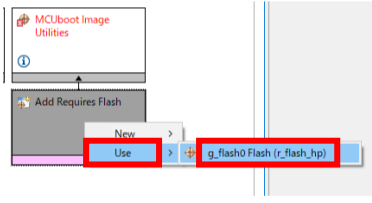
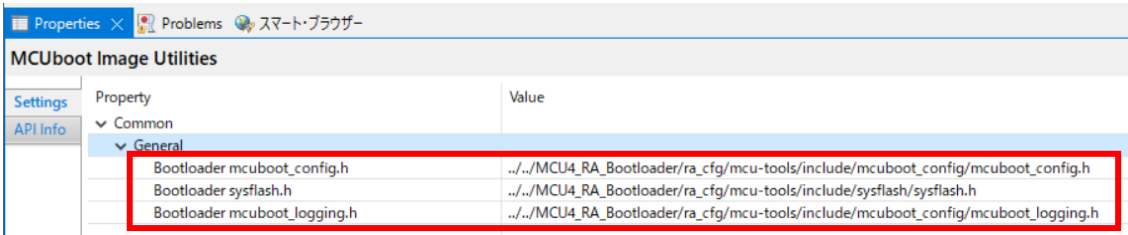
Also, to set the image_ok at run-time, the additional module “MCUboot Image Utilities” must be integrated into user application project, and then call boot_set_confirmed() API.

In this section, we will demonstrate the reverting of firmware.

6.1 Creating the project

Procedural Steps

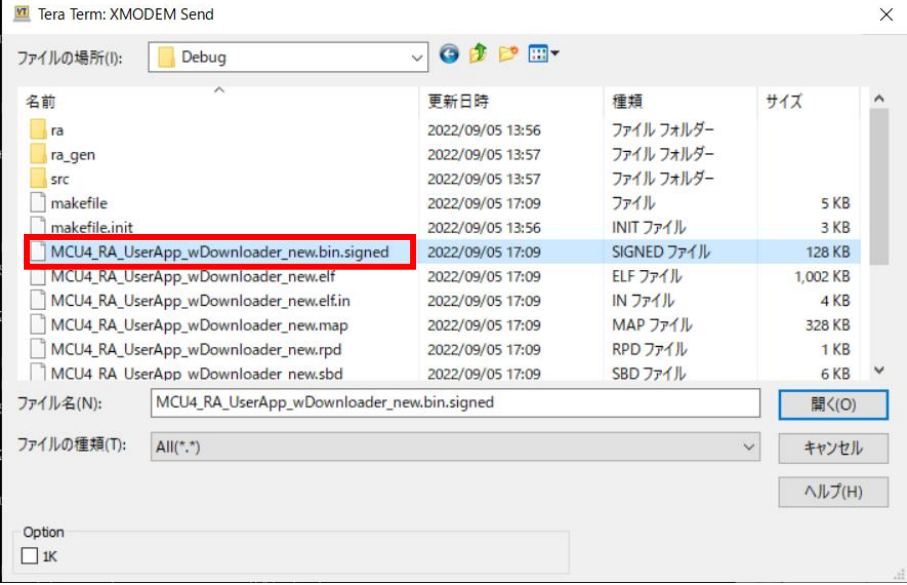
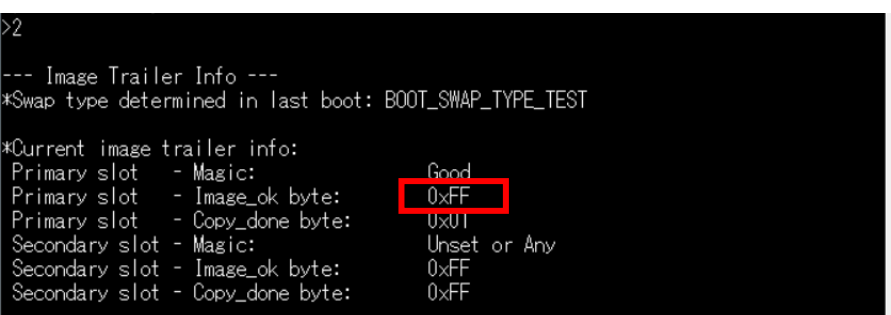
6.1.1	Before updating the firmware image in this section, please copy and keep existing firmware image file “MCU4_RA_UserApp_wDownloader.bin.signed” from project debug folder to local folder on PC. It will be used to MCU back to initial state. After copied, its file name would be better to change by adding any identifiable name. For example, “MCU4_RA_UserApp_wDownloader.bin.signed.initial”.
6.1.2	Open configuration.xml file of bootloader project “MCU4_RA_Bootloader”.
6.1.3	Configure the MCUboot stack. Common > Signing and Encryption Options > Custom: “--confirm” → “--pad” Note: Please do not forget a space “ ” before “-”. 
6.1.4	Press  to update the project code.
6.1.5	Build the bootloader project “MCU4_RA_Bootloader”.
6.1.6	Open configuration.xml file of user application project “MCU4_RA_UserApp_wDownloader_ new ”.

6.1.7	Add MCUboot Image Utilities module on Downloader Thread Stack. This module will use when set image_ok to 0x1. 
6.1.8	Add r_flash_hp driver to Requires Flash as Use. Configurator merges existing flash driver into this MCUboot Image Utilities module, but no warried. It is as expected. 
6.1.9	Configure the MCUboot Image Utilities module. - General > Bootloader mcuboot_config.h: “../MCU4_RA_Bootloader/ra_cfg/mcu-tools/include/mcuboot_config/mcuboot_config.h” - General > Bootloader sysflash.h: “../MCU4_RA_Bootloader/ra_cfg/mcu-tools/include/sysflash/sysflash.h” - General > Bootloader mcuboot_logging.h: “../MCU4_RA_Bootloader/ra_cfg/mcu-tools/include/mcuboot_config/mcuboot_logging.h” 
6.1.10	Build the user application project “MCU4_RA_UserApp_wDownloader_new”.

6.2 Revert to old application

Procedural Steps

6.2.1	To back to initial version of firmware image, please download the file “MCU4_RA_UserApp_wDownloader.bin.signed.initial” to MCU via Tera Term. The file MCU4_RA_UserApp_wDownloader.bin.signed.initial should have been copied to your local folder in Step 6.1.1. Then reset the MCU by typing “9” on Tera Term or pressing reset button.
-------	---

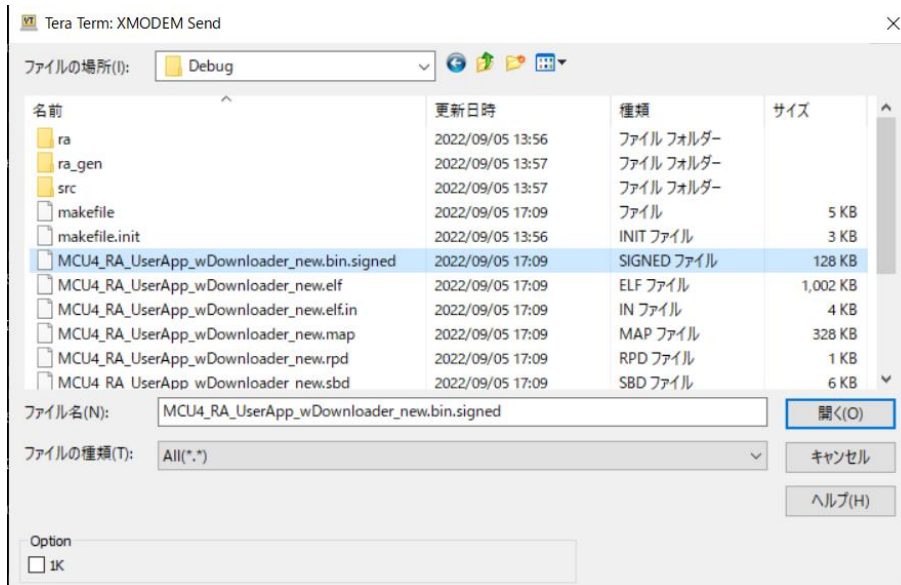
6.2.2	Type “4” and download “MCU4_RA_UserApp_wDownloader_new.bin.signed” that stored in user application project debug folder. 
6.2.3	Reset the MCU. Type “9” on Tera Term, or press the on-board Reset button. The MCUboot bootloader swaps the images. If image verification is ok, the user application in primary slot will start and the frequency of LED1(Blue) should change to high blink.
6.2.4	Type “2” to display the image trailer info. As current image_ok byte in primary slot is not 0x1, meaning this primary slot image is not confirmed. If user do not set 0x1 to the image_ok byte, bootloader will perform the reverting to old firmware using secondary slot image. In this section 6.2, in order to show the behavior of reverting, we will not update the image_ok byte. Bootloader will revert the firmware to old one.  <pre> >2 --- Image Trailer Info --- *Swap type determined in last boot: BOOT_SWAP_TYPE_TEST *Current image trailer info: Primary slot - Magic: Good Primary slot - Image_ok byte: 0xFF Primary slot - Copy_done byte: 0x01 Secondary slot - Magic: Unset or Any Secondary slot - Image_ok byte: 0xFF Secondary slot - Copy_done byte: 0xFF </pre>
6.2.5	Reset the MCU. Type “9” on Tera Term, or press the on-board Reset button. The frequency of LED1(Blue) should change to low blink. The MCUboot bootloader reverted the firmware.

6.3 Confirm new application not to revert to old application

Procedural Steps

6.3.1	To back to initial version of firmware image, please download the file “MCU4_RA_UserApp_wDownloader.bin.signed.initial” to MCU via Tera Term. Then reset the MCU by typing “9” on Tera Term or pressing reset button.
-------	---

- 6.3.2 Type “4” and download “MCU4_RA_UserApp_wDownloader_new.bin.signed” that stored in user application project debug folder.



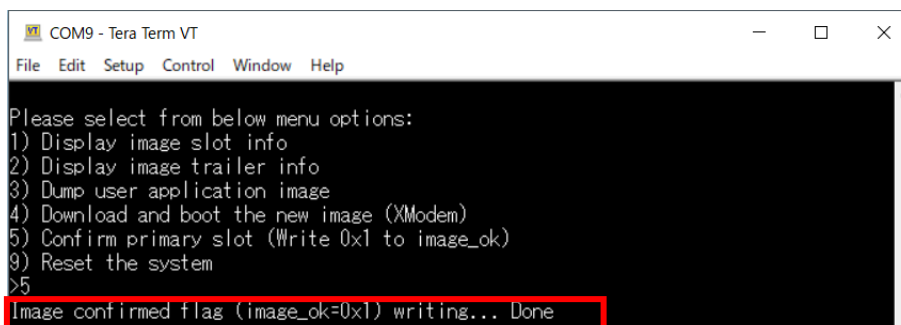
- 6.3.3 Reset the MCU. Type “9” on Tera Term, or press the on-board Reset button.
The MCUboot bootloader swaps the images. If ok, the user application in primary slot will start and the frequency of LED1(Blue) should change to **high blink**.

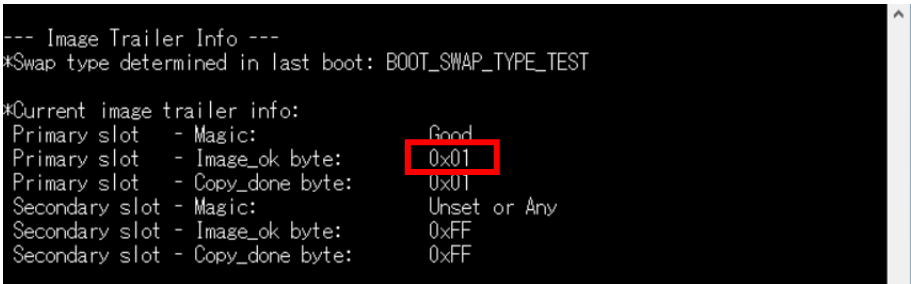
- 6.3.4 Type “2” to display the current and estimated next swap type.
As the image_ok in primary slot image is not 0x1, this image is not confirmed.

```
>2
--- Image Trailer Info ---
*Swap type determined in last boot: BOOT_SWAP_TYPE_TEST

*Current image trailer info:
Primary slot - Magic: Good
Primary slot - Image_ok byte: 0xFF
Primary slot - Copy_done byte: 0x01
Secondary slot - Magic: Unset or Any
Secondary slot - Image_ok byte: 0xFF
Secondary slot - Copy_done byte: 0xFF
```

- 6.3.5 This section 6.3 shows the permanent boot, so type “5” to set 0x1 to image_ok in new firmware. Bootloader will not do the reverting. New firmware is kept in the primary slot.



6.3.6	Type “2” again to show the trailer info. The image_ok byte may be changed to 0x1. It means that the new firmware has been confirmed and locked into the primary slot. It will permanently boot into this firmware until the new firmware is written to the secondary slot.  <pre> --- Image Trailer Info --- *Swap type determined in last boot: BOOT_SWAP_TYPE_TEST *Current image trailer info: Primary slot - Magic: Good Primary slot - Image_ok byte: 0x01 Primary slot - Copy_done byte: 0x01 Secondary slot - Magic: Unset or Any Secondary slot - Image_ok byte: 0xFF Secondary slot - Copy_done byte: 0xFF </pre>
6.3.7	Reset the MCU. Type “9” on Tera Term, or press the on-board Reset button. The frequency of LED1 (blue) should no unchanged and new user applications are held in the primary slot.
NOTE	From above results, we’ve done the demonstration of revert mechanism of MCUboot Swap Test mode. • Swap Test mode can revert the firmware to previous one. • To enable Swap Test mode, user should not set image_ok byte at build time. • RA configurator enables the image_ok setting configuration. To use it, change the bootloader project options from confirm-at-build-time “--confirm” to confirm-at-run-time “--pad”. • To set the image_ok byte at run-time, user should implement MCUboot Image Utilities module into user application project, and set 0x1 using boot_set_confirmed() API.

END OF SECTION

THANK YOU FOR ATTENDING THIS SESSION